Natalie Leung
014790932
9 November 2026

Data 260 Homework 9

Personal Configuration and Domain

SID4	0932
PORT_BASE	8032
PREFIX	s0932
SEED	0932
VERIFY_SEED	260932
DOMAIN_ID	4 (open-source package vulnerabilities)

Hardware: nvdia geforce 4070, msi cpu
Local model used: ollama qwen3:8b
Tagged commit hash repo with most updated codes (submission on canvas only):
https://github.com/n-k-leung/data260-0932

Part 1

Your Task

Code for redis backed counter

reports > hw09 > view_counter.py > run_test

```python
import redis
from sqlalchemy import inspect, text
from app.database import SessionLocal, engine
from app.models import Vul

PREFIX = "s0932"
class VulnerabilityDatabaseWithCache:
    def __init__(self, redis_host="localhost", redis_port=6379):
        self.db = SessionLocal()
        self.redis_client = redis.Redis(host=redis_host, port=redis_port, decode_responses=True)
        try:
            self.redis_client.ping()
            print("connected to redis")
        except redis.ConnectionError:
            print("failed to connect to Redis")
            raise

        self._setup_database()

    def _setup_database(self):
        columns = [column["name"] for column in inspect(engine).get_columns("vulnerabilities")]
        if "view_count" not in columns:
            with engine.begin() as conn:
                conn.execute(text("ALTER TABLE vulnerabilities ADD COLUMN view_count INT NOT NULL DEFAULT 0"))
            print("Added view_count column to the vulnerabilities table")

    def increment_vulnerability_views(self, vul_id):
        key = f"{PREFIX}:views:vulnerability:{vul_id}"
        count = self.redis_client.incr(key)
        #every 10 increments is saved to the database row
        if count % 10 == 0:
            vul = self.db.query(Vul).filter(Vul.id == vul_id).first()
            vul.view_count = count
            self.db.commit()
            print("✓ Synced views to database")
        return count

    def close(self):
        self.db.close()
        self.redis_client.close()

```

Output running docker

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09> docker compose up -d
[+] up 14/14
 ✓ Image redis/redis-stack-server:latest Pulled                                  21.2s
 ✓ Network hw09_default                  Created                                  0.1s
 ✓ Container s0932_redis                 Started                                  0.7s
```

Code for outputting counter for vulnerability testing incrementing

```python
def run_test():
    db = VulnerabilityDatabaseWithCache()
    #using first vul in the table for test
    vul = db.db.query(Vul).order_by(Vul.id.asc()).first()
    if vul is None:
        print("No vulnerabilities in the database to use")
        db.close()
        return
    vul_id = vul.id
    print(f"Testing vulnerability id={vul_id} (package: {vul.package_name})")

    #need to start counter from 0 so starts at 1
    db.redis_client.delete(f"{PREFIX}:views:vulnerability:{vul_id}")
    print("\n=== Testing Counter ===\n")
    for i in range(1, 26):
        total = db.increment_vulnerability_views(vul_id)
        print(f"Access #{i}: Total = {total}")
    db.db.refresh(vul)
    print(f"\ndatabase view_count for id={vul_id}: {vul.view_count}")
    print(f"redis value for {PREFIX}:views:vulnerability:{vul_id}: {db.redis_client.get(f'{PREFIX}:views:vulnerability:{vul_id}')}")

    db.close()

if __name__ == "__main__":
    run_test()

```

Output of testing counter

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09> python .\view_counter.
py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
connected to redis
Testing vulnerability id=1 (package: nodejs)

=== Testing Counter ===

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
✓ Synced views to database
Access #10: Total = 10
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
✓ Synced views to database
Access #20: Total = 20
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25

database view_count for id=1: 20
redis value for s0932:views:vulnerability:1: 25
```

Part 2: Semantic Response Cache

Assignment

Code for semantic cache implementation



```
reports > hw09 > semantic_cache.py > ...
50  class SemanticResponseCache:
51      def __init__(self, threshold=THRESHOLD):
52          self.threshold = threshold
53          self.redis_client = redis.Redis(host="localhost", port=6379, decode_responses=True)
54          self.redis_client.ping()
55          self.model = SentenceTransformer(EMBEDDING_MODEL)
56          self._create_index()
57
58      def _create_index(self):
59          try:
60              self.redis_client.ft(INDEX_NAME).info()
61          except redis.ResponseError:
62              self.redis_client.execute_command(
63                  "FT.CREATE", INDEX_NAME, "ON", "HASH", "PREFIX", "1", KEY_PREFIX,
64                  "SCHEMA",
65                  "query", "TEXT",
66                  "answer", "TEXT",
67                  "data_version", "TAG",
68                  "embedding", "VECTOR", "FLAT", "6",
69                  "TYPE", "FLOAT32", "DIM", EMBEDDING_DIM, "DISTANCE_METRIC", "COSINE",
70              )
71              log(f"Created vector index {INDEX_NAME}")
72
73      def embed(self, text):
74          return self.model.encode(text).astype(np.float32).tobytes()
75
76      def get_data_version(self):
77          self.redis_client.setnx(DATA_VERSION_KEY, 1)
78          return self.redis_client.get(DATA_VERSION_KEY)
79
80      def find_most_similar(self, embedding):
81          query = (
82              Query("*=>[KNN 1 @embedding $vec AS distance]")
83              .sort_by("distance")
84              .return_fields("query", "answer", "data_version", "distance")
85              .dialect(2)
86          )
87          results = self.redis_client.ft(INDEX_NAME).search(query, query_params={"vec": embedding})
88          if not results.docs:
89              return None, 0.0
90          best = results.docs[0]
91          similarity = 1 - float(best.distance)
92          return best, similarity
```



```
reports > hw09 > semantic_cache.py > ...
50  class SemanticResponseCache:
93
94      def store(self, question, embedding, answer):
95          key = KEY_PREFIX + str(uuid.uuid4())
96          self.redis_client.hset(key, mapping={
97              "query": question,
98              "embedding": embedding,
99              "answer": answer,
100             "data_version": self.get_data_version(),
101             "created_at": time.time(),
102         })
103         return key
104
105     def handle_query(self, question):
106         log(f"Start: {question}")
107         start = time.perf_counter()
108         embedding = self.embed(question)
109         best, similarity = self.find_most_similar(embedding)
110
111         if best is not None and similarity >= self.threshold:
112             latency_ms = (time.perf_counter() - start) * 1000
113             log(f"Cache hit, similarity {similarity:.3f}, matched: {best.query}")
114             log(f"End: {latency_ms:.1f} ms")
115             return {
116                 "question": question,
117                 "answer": best.answer,
118                 "is_cached": True,
119                 "similarity": round(similarity, 3),
120                 "cached_query": best.query,
121                 "data_version": best.data_version,
122                 "latency_ms": round(latency_ms, 1),
123             }
124         log(f"Cache miss, best similarity {similarity:.3f}, asking the agents")
125         result = asyncio.run(ask_agents(question))
126         answer = result["answer"]
127         self.store(question, embedding, answer)
128
129         latency_ms = (time.perf_counter() - start) * 1000
130         log(f"End: {latency_ms:.1f} ms")
131         return {
132             "question": question,
133             "answer": answer,
134             "is_cached": False,
135             "similarity": round(similarity, 3),
136             "cached_query": None,
137             "data_version": self.get_data_version(),
138             "latency_ms": round(latency_ms, 1),
139         }
140
141     def clear(self):
142         try:
143             self.redis_client.ft(INDEX_NAME).dropindex(delete_documents=True)
144         except redis.ResponseError:
145             pass
146         self._create_index()
```

Code for miss path through hw8 agents

```
24  async def ask_agents(question):
25      correlation_id = str(uuid.uuid4())
26      consumer = AIOKafkaConsumer(FINAL_TOPIC, bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS, auto_offset_reset="earliest")
27      producer = AIOKafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS)
28      await consumer.start()
29      await producer.start()
30
31      try:
32          await producer.send_and_wait(INBOX_TOPIC, to_bytes({"correlation_id": correlation_id, "question": question}))
33          log(f"Sent the question to {INBOX_TOPIC} (correlation_id {correlation_id})")
34
35          #skip older results
36          async def wait_for_result():
37              async for msg in consumer:
38                  result = from_bytes(msg.value)
39                  if result.get("correlation_id") == correlation_id:
40                      return result
41
42          result = await asyncio.wait_for(wait_for_result(), timeout=AGENT_TIMEOUT_SECONDS)
43          log(f"Got the answer from {FINAL_TOPIC} (correlation_id {correlation_id}, status {result.get('status')})")
44          return result
45      finally:
46          await consumer.stop()
47          await producer.stop()
```

Code to test at least 10 queries

```python
QUERIES = [
    ("new", "What should our team do about CVE-2021-23337 in lodash 4.17.20?"),
    ("exact duplicate", "What should our team do about CVE-2021-23337 in lodash 4.17.20?"),
    ("paraphrase", "How should we handle the lodash 4.17.20 vulnerability CVE-2021-23337?"),
    ("new", "Which version of requests fixes CVE-2018-18074?"),
    ("exact duplicate", "Which version of requests fixes CVE-2018-18074?"),
    ("paraphrase", "What requests release patches CVE-2018-18074?"),
    ("new", "What is a software bill of materials (SBOM)?"),
    ("paraphrase", "Can you explain what an SBOM is?"),
    ("new", "How often should we scan our open source dependencies for vulnerabilities?"),
    ("exact duplicate", "How often should we scan our open source dependencies for vulnerabilities?"),
    ("new", "What does a CVSS score of 9.8 mean?"),
    ("paraphrase", "Explain what a CVSS rating of 9.8 means."),
]

def main():
    cache = SemanticResponseCache(threshold=0.85)
    #have to clear cache so can rerun and get same results
    cache.clear()

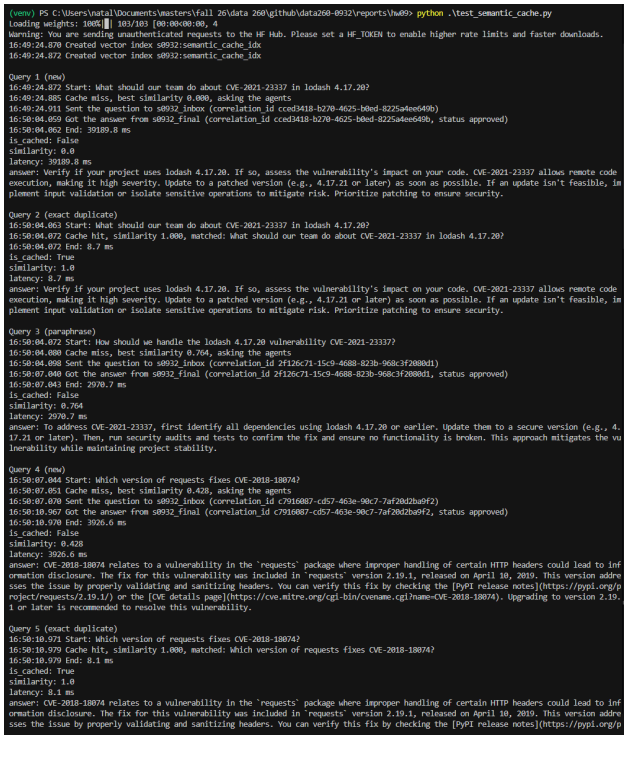
    results = []
    for number, (query_type, question) in enumerate(QUERIES, start=1):
        print(f"\nQuery {number} ({query_type})")
        response = cache.handle_query(question)
        results.append((query_type, response))
        print(f"is_cached: {response['is_cached']}")
        print(f"similarity: {response['similarity']}")
        print(f"latency: {response['latency_ms']} ms")
        print(f"answer: {response['answer']}")
```

Output of agent planner, writer, and reviewer



```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\reviewer.py
16:50:03.298 REVIEWER RECEIVE from s0932_drafts partition=0 offset=0 correlation_id=cced3418-b270-4625-b0ed-8225a4ee649b
16:50:04.058 REVIEWER SEND to s0932_final partition=0 offset=0 correlation_id=cced3418-b270-4625-b0ed-8225a4ee649b
16:50:04.058 REVIEWER status=approved review=APPROVED
The draft clearly outlines the necessary steps to address CVE-2021-23337 in lodash 4.17.20, including verification, impact assessment, mitiga
tion strategies, and prioritization of patching.
16:50:06.511 REVIEWER RECEIVE from s0932_drafts partition=0 offset=1 correlation_id=2f126c71-15c9-4688-823b-968c3f2080d1
16:50:07.039 REVIEWER SEND to s0932_final partition=0 offset=1 correlation_id=2f126c71-15c9-4688-823b-968c3f2080d1
16:50:07.039 REVIEWER status=approved review=APPROVED
The draft clearly outlines the steps to address the CVE-2021-23337 vulnerability by updating dependencies and verifying the fix.
16:50:10.366 REVIEWER RECEIVE from s0932_drafts partition=0 offset=2 correlation_id=c7916087-cd57-463e-90c7-7af20d2ba9f2
16:50:10.967 REVIEWER SEND to s0932_final partition=0 offset=2 correlation_id=c7916087-cd57-463e-90c7-7af20d2ba9f2
16:50:10.967 REVIEWER status=approved review=APPROVED
The draft correctly identifies the version of `requests` that fixes CVE-2018-18074 and provides accurate information about the fix and recomm
ended action.
16:50:13.493 REVIEWER RECEIVE from s0932_drafts partition=0 offset=3 correlation_id=9a349cfa-61d3-4f34-baaf-dea29f1f7c19
16:50:14.053 REVIEWER SEND to s0932_final partition=0 offset=3 correlation_id=9a349cfa-61d3-4f34-baaf-dea29f1f7c19
16:50:14.053 REVIEWER status=approved review=APPROVED
The draft correctly identifies the origin of the patch request for CVE-2018-18074 and explains the context of the vulnerability and its resol
ution.
16:50:15.976 REVIEWER RECEIVE from s0932_drafts partition=0 offset=4 correlation_id=1c4fc4f6-672b-4578-9001-8b40a13717f9
16:50:16.396 REVIEWER SEND to s0932_final partition=0 offset=4 correlation_id=1c4fc4f6-672b-4578-9001-8b40a13717f9
16:50:16.396 REVIEWER status=approved review=APPROVED
The draft clearly and accurately defines an SBOM, explaining its purpose, components, and benefits in a concise manner.
16:50:18.669 REVIEWER RECEIVE from s0932_drafts partition=0 offset=5 correlation_id=1cda0aad-88d8-4d21-b6d2-b6c1dbfeb9f7
16:50:19.159 REVIEWER SEND to s0932_final partition=0 offset=5 correlation_id=1cda0aad-88d8-4d21-b6d2-b6c1dbfeb9f7
16:50:19.159 REVIEWER status=approved review=APPROVED
The draft clearly and accurately explains what an SBOM is, its purpose, common formats, and its importance in software security and supply ch
ain management.
16:50:21.053 REVIEWER RECEIVE from s0932_drafts partition=0 offset=6 correlation_id=4b11efe4-28fb-4274-9587-03b670d3873d
16:50:21.515 REVIEWER SEND to s0932_final partition=0 offset=6 correlation_id=4b11efe4-28fb-4274-9587-03b670d3873d
16:50:21.515 REVIEWER status=approved review=APPROVED
The draft clearly answers the question by providing a structured approach to scanning frequency based on project needs and recommends tools a
nd policies for maintaining security.
16:50:23.970 REVIEWER RECEIVE from s0932_drafts partition=0 offset=7 correlation_id=2f9470bf-d7b4-49ce-844c-17141b549787
16:50:24.432 REVIEWER SEND to s0932_final partition=0 offset=7 correlation_id=2f9470bf-d7b4-49ce-844c-17141b549787
16:50:24.432 REVIEWER status=approved review=APPROVED
The draft accurately explains the significance of a CVSS 9.8 score, its implications, and the urgency of remediation.
```

Output to test system with at least 10 queries



```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09> python .\test_semantic_cache.py
Loading weights: 100%|| 103/103 [00:00<00:00, 4
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
16:49:24.870 Created vector index s0932:semantic_cache_idx
16:49:24.872 Created vector index s0932:semantic_cache_idx

Query 1 (new)
16:49:24.872 Start: What should our team do about CVE-2021-23337 in lodash 4.17.20?
16:49:24.885 Cache miss, best similarity 0.000, asking the agents
16:49:24.911 Sent the question to s0932_inbox (correlation_id cced3418-b270-4625-b0ed-8225a4ee649b)
16:50:04.059 Got the answer from s0932_final (correlation_id cced3418-b270-4625-b0ed-8225a4ee649b, status approved)
16:50:04.062 End: 39189.8 ms
is_cached: False
similarity: 0.0
latency: 39180.8 ms
answer: Verify if your project uses lodash 4.17.20. If so, assess the vulnerability's impact on your code. CVE-2021-23337 allows remote code
 execution, making it high severity. Update to a patched version (e.g., 4.17.21 or later) as soon as possible. If an update isn't feasible, im
plement input validation or isolate sensitive operations to mitigate risk. Prioritize patching to ensure security.

Query 2 (exact duplicate)
16:50:04.063 Start: What should our team do about CVE-2021-23337 in lodash 4.17.20?
16:50:04.072 Cache hit, similarity 1.000, matched: What should our team do about CVE-2021-23337 in lodash 4.17.20?
16:50:04.072 End: 8.7 ms
is_cached: True
similarity: 1.0
latency: 8.7 ms
answer: Verify if your project uses lodash 4.17.20. If so, assess the vulnerability's impact on your code. CVE-2021-23337 allows remote code
 execution, making it high severity. Update to a patched version (e.g., 4.17.21 or later) as soon as possible. If an update isn't feasible, im
plement input validation or isolate sensitive operations to mitigate risk. Prioritize patching to ensure security.

Query 3 (paraphrase)
16:50:04.072 Start: How should we handle the lodash 4.17.20 vulnerability CVE-2021-23337?
16:50:04.080 Cache miss, best similarity 0.764, asking the agents
16:50:04.098 Sent the question to s0932_inbox (correlation_id 2f126c71-15c9-4688-823b-968c3f2080d1)
16:50:07.040 Got the answer from s0932_final (correlation_id 2f126c71-15c9-4688-823b-968c3f2080d1, status approved)
16:50:07.043 End: 2970.7 ms
is_cached: False
similarity: 0.764
latency: 2970.7 ms
answer: To address CVE-2021-23337, first identify all dependencies using lodash 4.17.20 or earlier. Update them to a secure version (e.g., 4.
17.21 or later). Then, run security audits and tests to confirm the fix and ensure no functionality is broken. This approach mitigates the vu
lnerability while maintaining project stability.

Query 4 (new)
16:50:07.044 Start: Which version of requests fixes CVE-2018-18074?
16:50:07.051 Cache miss, best similarity 0.428, asking the agents
16:50:07.070 Sent the question to s0932_inbox (correlation_id c7916087-cd57-463e-90c7-7af20d2ba9f2)
16:50:10.967 Got the answer from s0932_final (correlation_id c7916087-cd57-463e-90c7-7af20d2ba9f2, status approved)
16:50:10.970 End: 3926.6 ms
is_cached: False
similarity: 0.428
latency: 3926.6 ms
answer: CVE-2018-18074 relates to a vulnerability in the `requests` package where improper handling of certain HTTP headers could lead to inf
ormation disclosure. The fix for this vulnerability was included in `requests` version 2.19.1, released on April 10, 2019. This version addre
sses the issue by properly validating and sanitizing headers. You can verify this fix by checking the [PyPI release notes](https://pypi.org/p
roject/requests/2.19.1/) or the [CVE details page](https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-18074). Upgrading to version 2.19.
1 or later is recommended to resolve this vulnerability.

Query 5 (exact duplicate)
16:50:10.971 Start: Which version of requests fixes CVE-2018-18074?
16:50:10.979 Cache hit, similarity 1.000, matched: Which version of requests fixes CVE-2018-18074?
16:50:10.979 End: 8.1 ms
is_cached: True
similarity: 1.0
latency: 8.1 ms
answer: CVE-2018-18074 relates to a vulnerability in the `requests` package where improper handling of certain HTTP headers could lead to inf
ormation disclosure. The fix for this vulnerability was included in `requests` version 2.19.1, released on April 10, 2019. This version addre
sses the issue by properly validating and sanitizing headers. You can verify this fix by checking the [PyPI release notes](https://pypi.org/p
roject/requests/2.19.1/) or the [CVE details page](https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-18074). Upgrading to version 2.19.
1 or later is recommended to resolve this vulnerability.

Query 6 (paraphrase)
16:50:10.980 Start: What requests release patches CVE-2018-18074?
16:50:10.987 Cache miss, best similarity 0.788, asking the agents
16:50:11.006 Sent the question to s0932_inbox (correlation_id 9a349cfa-61d3-4f34-baaf-dea29f1f7c19)
16:50:14.054 Got the answer from s0932_final (correlation_id 9a349cfa-61d3-4f34-baaf-dea29f1f7c19, status approved)
16:50:14.057 End: 3077.2 ms
is_cached: False
similarity: 0.788
latency: 3077.2 ms
answer: CVE-2018-18074 affects the Linux kernel's `ip6gre` module. The patch was released by the Linux kernel maintainers, specifically Linus
 Torvalds and the kernel development team. The request for the patch originated from a vulnerability report by the National Vulnerability Dat
abase (NVD) and was likely triggered by security researchers or users encountering the issue. The official request was not publicly detailed,
 but the patch was prompted by the discovery of a buffer overflow vulnerability in the IPv6 GRE tunneling code.

Query 7 (new)
16:50:14.058 Start: What is a software bill of materials (SBOM)?
16:50:14.067 Cache miss, best similarity 0.262, asking the agents
16:50:14.085 Sent the question to s0932_inbox (correlation_id 1c4fc4f6-672b-4578-9001-8b40a13717f9)
16:50:16.396 Got the answer from s0932_final (correlation_id 1c4fc4f6-672b-4578-9001-8b40a13717f9, status approved)
16:50:16.399 End: 2341.2 ms
is_cached: False
similarity: 0.262
latency: 2341.2 ms
answer: An SBOM is a formal document listing all the components, libraries, and dependencies in a software product. Its purpose is to provide
 transparency and visibility into the software supply chain. Typical components include software names, versions, licenses, and cryptographic
 hashes. Using an SBOM enhances vulnerability management by enabling quick identification and remediation of security issues, ensuring compli
ance, and supporting incident response. It also aids in tracking dependencies and reducing the risk of using outdated or insecure components.

Query 8 (paraphrase)
16:50:16.400 Start: Can you explain what an SBOM is?
16:50:16.408 Cache miss, best similarity 0.601, asking the agents
16:50:16.427 Sent the question to s0932_inbox (correlation_id 1cda0aad-88d8-4d21-b6d2-b6c1dbfeb9f7)
16:50:19.160 Got the answer from s0932_final (correlation_id 1cda0aad-88d8-4d21-b6d2-b6c1dbfeb9f7, status approved)
16:50:19.162 End: 2762.4 ms
is_cached: False
similarity: 0.601
latency: 2762.4 ms
answer: An SBOM (Software Bill of Materials) is a formal document listing all components, libraries, and dependencies in a software product.
Its purpose is to enhance transparency and security by providing visibility into software composition. Common SBOM formats include SPDX, Cycl
oneDX, and SWID, each offering structured data for component tracking. SBOMs help identify and manage vulnerabilities by enabling rapid detec
tion of outdated or insecure components, facilitating patching and compliance efforts. They are essential for secure software development and
 supply chain risk management.

Query 9 (new)
16:50:19.163 Start: How often should we scan our open source dependencies for vulnerabilities?
16:50:19.172 Cache miss, best similarity 0.374, asking the agents
16:50:19.190 Sent the question to s0932_inbox (correlation_id 4b11efe4-28fb-4274-9587-03b670d3873d)
16:50:21.515 Got the answer from s0932_final (correlation_id 4b11efe4-28fb-4274-9587-03b670d3873d, status approved)
16:50:21.518 End: 2354.8 ms
is_cached: False
similarity: 0.374
latency: 2354.8 ms
answer: Scan your open source dependencies for vulnerabilities regularly, ideally with each code update or release. If your project has a fre
quent release cycle, scan daily or weekly. For less frequent updates, monthly scans may suffice. Use automated tools like Dependabot or Snyk
 to enable real-time or scheduled scanning. Establish a policy to address critical vulnerabilities immediately and review lower-severity issue
s periodically. This ensures your dependencies remain secure and up-to-date.

Query 10 (exact duplicate)
16:50:21.519 Start: How often should we scan our open source dependencies for vulnerabilities?
16:50:21.527 Cache hit, similarity 1.000, matched: How often should we scan our open source dependencies for vulnerabilities?
16:50:21.527 End: 7.7 ms
is_cached: True
similarity: 1.0
latency: 7.7 ms
answer: Scan your open source dependencies for vulnerabilities regularly, ideally with each code update or release. If your project has a fre
```

```
quent release cycle, scan daily or weekly. For less frequent updates, monthly scans may suffice. Use automated tools like Dependabot or Snyk
to enable real-time or scheduled scanning. Establish a policy to address critical vulnerabilities immediately and review lower-severity issue
s periodically. This ensures your dependencies remain secure and up-to-date.

Query 11 (new)
16:50:21.527 Start: What does a CVSS score of 9.8 mean?
16:50:21.535 Cache miss, best similarity 0.370, asking the agents
16:50:21.552 Sent the question to s0932_inbox (correlation_id 2f9470bf-d7b4-49ce-844c-17141b549787)
16:50:24.432 Got the answer from s0932_final (correlation_id 2f9470bf-d7b4-49ce-844c-17141b549787, status approved)
16:50:24.435 End: 2908.3 ms
is_cached: False
similarity: 0.37
latency: 2908.3 ms
answer: A CVSS score of 9.8 indicates a critical severity level, as it is the highest score (out of 10) and signifies a high-impact, easily e
xploitable vulnerability. This score typically results from factors like high exploitability, significant impact on confidentiality, integrit
y, and availability, and minimal required user interaction. A 9.8 score means the vulnerability should be prioritized for immediate remediati
on to prevent potential system compromise and data loss. Organizations must address such vulnerabilities swiftly to maintain security and min
imize risk.

Query 12 (paraphrase)
16:50:24.436 Start: Explain what a CVSS rating of 9.8 means.
16:50:24.444 Cache hit, similarity 0.946, matched: What does a CVSS score of 9.8 mean?
16:50:24.444 End: 7.4 ms
is_cached: True
similarity: 0.946
latency: 7.4 ms
answer: A CVSS score of 9.8 indicates a critical severity level, as it is the highest score (out of 10) and signifies a high-impact, easily e
xploitable vulnerability. This score typically results from factors like high exploitability, significant impact on confidentiality, integrit
y, and availability, and minimal required user interaction. A 9.8 score means the vulnerability should be prioritized for immediate remediati
on to prevent potential system compromise and data loss. Organizations must address such vulnerabilities swiftly to maintain security and min
imize risk.

Results
1. new, is_cached: False, similarity: 0.0, latency: 39189.8 ms, question: What should our team do about CVE-2021-23337 in lodash 4.17.20?
2. exact duplicate, is_cached: True, similarity: 1.0, latency: 8.7 ms, question: What should our team do about CVE-2021-23337 in lodash 4.17.
20?
3. paraphrase, is_cached: False, similarity: 0.764, latency: 2970.7 ms, question: How should we handle the lodash 4.17.20 vulnerability CVE-2
021-23337?
4. new, is_cached: False, similarity: 0.428, latency: 3926.6 ms, question: Which version of requests fixes CVE-2018-18074?
5. exact duplicate, is_cached: True, similarity: 1.0, latency: 8.1 ms, question: Which version of requests fixes CVE-2018-18074?
6. paraphrase, is_cached: False, similarity: 0.788, latency: 3077.2 ms, question: What requests release patches CVE-2018-18074?
7. new, is_cached: False, similarity: 0.262, latency: 2341.2 ms, question: What is a software bill of materials (SBOM)?
8. paraphrase, is_cached: False, similarity: 0.601, latency: 2762.4 ms, question: Can you explain what an SBOM is?
9. new, is_cached: False, similarity: 0.374, latency: 2354.8 ms, question: How often should we scan our open source dependencies for vulnerab
ilities?
10. exact duplicate, is_cached: True, similarity: 1.0, latency: 7.7 ms, question: How often should we scan our open source dependencies for v
ulnerabilities?
11. new, is_cached: False, similarity: 0.37, latency: 2908.3 ms, question: What does a CVSS score of 9.8 mean?
12. paraphrase, is_cached: True, similarity: 0.946, latency: 7.4 ms, question: Explain what a CVSS rating of 9.8 means.

Cache hit rate: 4 of 12 (33.3%)
Average latency, cached: 8.0 ms
Average latency, not cached: 7441.4 ms
(venv) PS C:\Users\natal\Documents\masters\fa□l 26\data 260\github\data260-0932\reports\hw09>
```

Measuring Cache Performance

Code for measuring cache performance (output for this is shown above)

```python
    def handle_query(self, question):
        log(f"Start: {question}")
        start = time.perf_counter()
        embedding = self.embed(question)
        best, similarity = self.find_most_similar(embedding)

        if best is not None and similarity >= self.threshold:
            latency_ms = (time.perf_counter() - start) * 1000
            log(f"Cache hit, similarity {similarity:.3f}, matched: {best.query}")
            log(f"End: {latency_ms:.1f} ms")
            return {
                "question": question,
                "answer": best.answer,
                "is_cached": True,
                "similarity": round(similarity, 3),
                "cached_query": best.query,
                "data_version": best.data_version,
                "latency_ms": round(latency_ms, 1),
            }
        log(f"Cache miss, best similarity {similarity:.3f}, asking the agents")
        result = asyncio.run(ask_agents(question))
        answer = result["answer"]
        self.store(question, embedding, answer)

        latency_ms = (time.perf_counter() - start) * 1000
        log(f"End: {latency_ms:.1f} ms")
        return {
            "question": question,
            "answer": answer,
            "is_cached": False,
            "similarity": round(similarity, 3),
            "cached_query": None,
            "data_version": self.get_data_version(),
            "latency_ms": round(latency_ms, 1),
        }

```

Output Format

Code for output format

reports > hw09 > test_semantic_cache.py > main

```python

def main():
    cache = SemanticResponseCache(threshold=0.85)
    #have to clear cache so can rerun and get same results
    cache.clear()

    results = []
    for number, (query_type, question) in enumerate(QUERIES, start=1):
        print(f"\nQuery {number} ({query_type})")
        response = cache.handle_query(question)
        results.append((query_type, response))
        print(f"is_cached: {response['is_cached']}")
        print(f"similarity: {response['similarity']}")
        print(f"latency: {response['latency_ms']} ms")
        print(f"answer: {response['answer']}")

    print("\nResults")
    for number, (query_type, r) in enumerate(results, start=1):
        print(f"{number}. {query_type}, is_cached: {r['is_cached']}, similarity: {r['similarity']}, latency: {r['latency_ms']} ms, question: {r['question']}")
    cached = [r for _, r in results if r["is_cached"]]
    not_cached = [r for _, r in results if not r["is_cached"]]
    hit_rate = len(cached) / len(results) * 100
    print(f"\nCache hit rate: {len(cached)} of {len(results)} ({hit_rate:.1f}%)")
    if cached:
        print(f"Average latency, cached: {sum(r['latency_ms'] for r in cached) / len(cached):.1f} ms")
    if not_cached:
        print(f"Average latency, not cached: {sum(r['latency_ms'] for r in not_cached) / len(not_cached):.1f} ms")

if __name__ == "__main__":
    main()
```

Output of reported hit rate and latency

```
Results
1. new, is_cached: False, similarity: 0.0, latency: 39189.8 ms, question: What should our team do about CVE-2021-23337 in lodash 4.17.20?
2. exact duplicate, is_cached: True, similarity: 1.0, latency: 8.7 ms, question: What should our team do about CVE-2021-23337 in lodash 4.17.
20?
3. paraphrase, is_cached: False, similarity: 0.764, latency: 2970.7 ms, question: How should we handle the lodash 4.17.20 vulnerability CVE-2
021-23337?
4. new, is_cached: False, similarity: 0.428, latency: 3926.6 ms, question: Which version of requests fixes CVE-2018-18074?
5. exact duplicate, is_cached: True, similarity: 1.0, latency: 8.1 ms, question: Which version of requests fixes CVE-2018-18074?
6. paraphrase, is_cached: False, similarity: 0.788, latency: 3077.2 ms, question: What requests release patches CVE-2018-18074?
7. new, is_cached: False, similarity: 0.262, latency: 2341.2 ms, question: What is a software bill of materials (SBOM)?
8. paraphrase, is_cached: False, similarity: 0.601, latency: 2762.4 ms, question: Can you explain what an SBOM is?
9. new, is_cached: False, similarity: 0.374, latency: 2354.8 ms, question: How often should we scan our open source dependencies for vulnerab
ilities?
10. exact duplicate, is_cached: True, similarity: 1.0, latency: 7.7 ms, question: How often should we scan our open source dependencies for v
ulnerabilities?
11. new, is_cached: False, similarity: 0.37, latency: 2908.3 ms, question: What does a CVSS score of 9.8 mean?
12. paraphrase, is_cached: True, similarity: 0.946, latency: 7.4 ms, question: Explain what a CVSS rating of 9.8 means.

Cache hit rate: 4 of 12 (33.3%)
Average latency, cached: 8.0 ms
Average latency, not cached: 7441.4 ms
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09>
```

Part 3: Threshold Economics and Stale Answers

3.1 Labeled test set

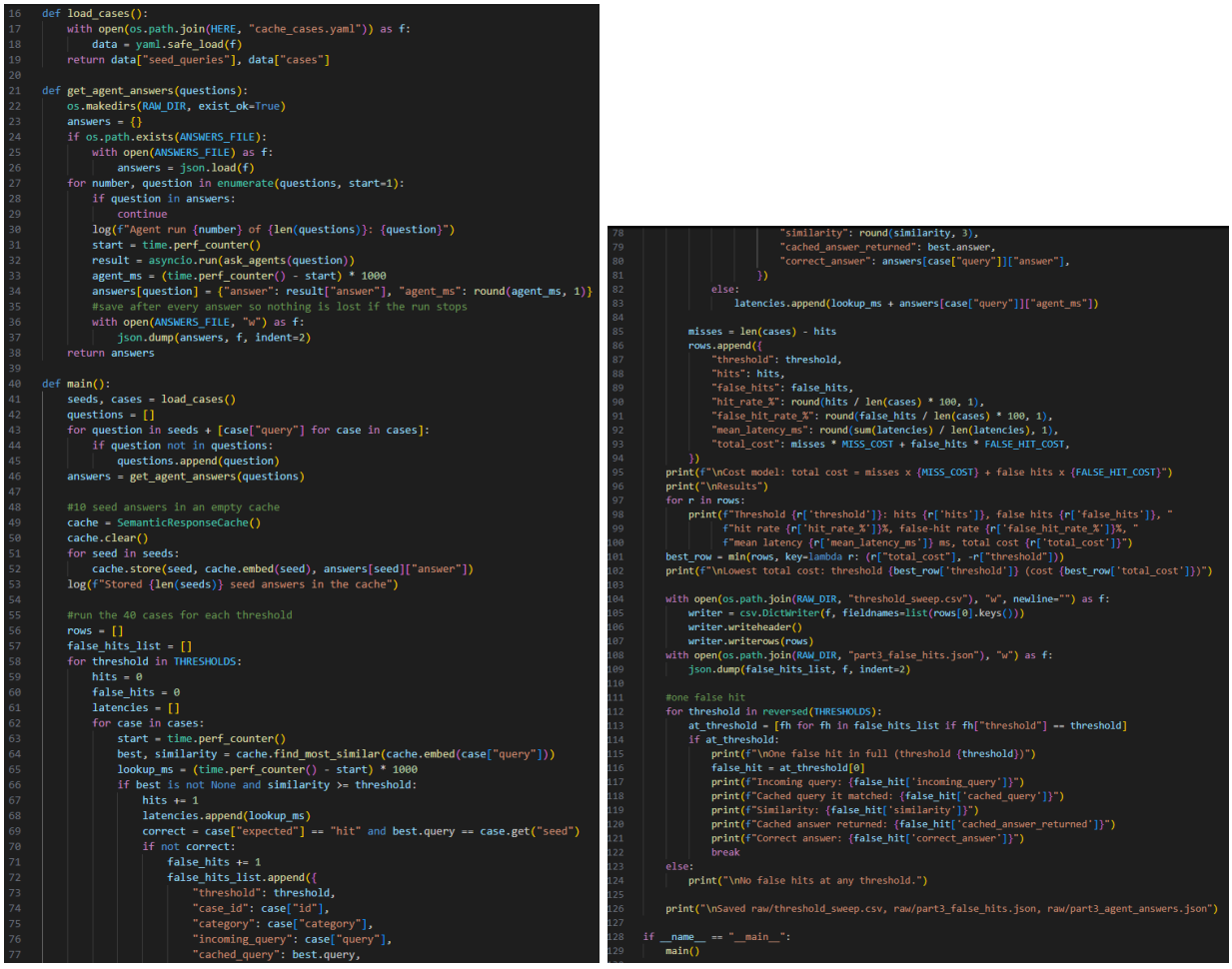
Cache_cases.yaml

```
> hw09 > ! cache_cases.yaml
seed_queries:
  - "What should our team do about CVE-2021-23337 in lodash 4.17.20?"
  - "Which version of requests fixes CVE-2018-18074?"
  - "What is the CVSS severity rating for a score of 9.8?"
  - "What is a software bill of materials (SBOM)?"
  - "Which log4j version fixes the Log4Shell vulnerability CVE-2021-44228?"
  - "What is a transitive dependency vulnerability?"
  - "How do we check if our npm project uses a vulnerable package?"
  - "What type of vulnerability is prototype pollution?"
  - "How quickly should we patch a Critical severity vulnerability?"
  - "What is the difference between a CVE and a CWE?"

cases:
  - {id: 1, category: exact_duplicate, expected: hit, query: "What should our team do about CVE-2021-23337 in lodash 4.17.20?", seed: "What should our team do about CVE-2021-23337 in lodash 4.17.20?"}
  - {id: 2, category: exact_duplicate, expected: hit, query: "Which version of requests fixes CVE-2018-18074?", seed: "Which version of requests fixes CVE-2018-18074?"}
  - {id: 3, category: exact_duplicate, expected: hit, query: "What is the CVSS severity rating for a score of 9.8?", seed: "What is the CVSS severity rating for a score of 9.8?"}
  - {id: 4, category: exact_duplicate, expected: hit, query: "What is a software bill of materials (SBOM)?", seed: "What is a software bill of materials (SBOM)?"}
  - {id: 5, category: exact_duplicate, expected: hit, query: "Which log4j version fixes the Log4Shell vulnerability CVE-2021-44228?", seed: "Which log4j version fixes the Log4Shell vulnerability CVE-2021-44228?"}
  - {id: 6, category: exact_duplicate, expected: hit, query: "What is a transitive dependency vulnerability?", seed: "What is a transitive dependency vulnerability?"}
  - {id: 7, category: exact_duplicate, expected: hit, query: "How do we check if our npm project uses a vulnerable package?", seed: "How do we check if our npm project uses a vulnerable package?"}
  - {id: 8, category: exact_duplicate, expected: hit, query: "What type of vulnerability is prototype pollution?", seed: "What type of vulnerability is prototype pollution?"}
  - {id: 9, category: exact_duplicate, expected: hit, query: "How quickly should we patch a Critical severity vulnerability?", seed: "How quickly should we patch a Critical severity vulnerability?"}
  - {id: 10, category: exact_duplicate, expected: hit, query: "What is the difference between a CVE and a CWE?", seed: "What is the difference between a CVE and a CWE?"}
  - {id: 11, category: paraphrase, expected: hit, query: "How should we respond to CVE-2021-23337 affecting lodash 4.17.20?", seed: "What should our team do about CVE-2021-23337 in lodash 4.17.20?"}
  - {id: 12, category: paraphrase, expected: hit, query: "What requests release patches CVE-2018-18074?", seed: "Which version of requests fixes CVE-2018-18074?"}
  - {id: 13, category: paraphrase, expected: hit, query: "Which severity level does a CVSS score of 9.8 fall into?", seed: "What is the CVSS severity rating for a score of 9.8?"}
  - {id: 14, category: paraphrase, expected: hit, query: "Can you explain what an SBOM is?", seed: "What is a software bill of materials (SBOM)?"}
  - {id: 15, category: paraphrase, expected: hit, query: "What log4j version should we upgrade to for Log4Shell (CVE-2021-44228)?", seed: "Which log4j version fixes the Log4Shell vulnerability CVE-2021-44228?"}
  - {id: 16, category: paraphrase, expected: hit, query: "What does it mean when a vulnerability is in a transitive dependency?", seed: "What is a transitive dependency vulnerability?"}
  - {id: 17, category: paraphrase, expected: hit, query: "How can we find vulnerable packages in our npm project?", seed: "How do we check if our npm project uses a vulnerable package?"}
  - {id: 18, category: paraphrase, expected: hit, query: "Can you explain the prototype pollution vulnerability type?", seed: "What type of vulnerability is prototype pollution?"}
  - {id: 19, category: paraphrase, expected: hit, query: "What is the recommended time to fix a Critical vulnerability?", seed: "How quickly should we patch a Critical severity vulnerability?"}
  - {id: 20, category: paraphrase, expected: hit, query: "How is a CVE different from a CWE?", seed: "What is the difference between a CVE and a CWE?"}
  - {id: 21, category: similar_different, expected: miss, query: "What should our team do about CVE-2020-8203 in lodash 4.17.15?", seed: "What should our team do about CVE-2021-23337 in lodash 4.17.20?"}
  - {id: 22, category: similar_different, expected: miss, query: "Which version of requests fixes CVE-2023-32681?", seed: "Which version of requests fixes CVE-2018-18074?"}
  - {id: 23, category: similar_different, expected: miss, query: "What is the CVSS severity rating for a score of 5.3?", seed: "What is the CVSS severity rating for a score of 9.8?"}
  - {id: 24, category: similar_different, expected: miss, query: "What is a vulnerability exploitability exchange (VEX) document?", seed: "What is a software bill of materials (SBOM)?"}
  - {id: 25, category: similar_different, expected: miss, query: "Which log4j version fixes CVE-2021-45046?", seed: "Which log4j version fixes the Log4Shell vulnerability CVE-2021-44228?"}
  - {id: 26, category: similar_different, expected: miss, query: "What is a direct dependency vulnerability?", seed: "What is a transitive dependency vulnerability?"}
  - {id: 27, category: similar_different, expected: miss, query: "How do we check if our Python project uses a vulnerable package?", seed: "How do we check if our npm project uses a vulnerable package?"}
  - {id: 28, category: similar_different, expected: miss, query: "What type of vulnerability is a regular expression denial of service (ReDoS)?", seed: "What type of vulnerability is prototype pollution?"}
  - {id: 29, category: similar_different, expected: miss, query: "How quickly should we patch a Low severity vulnerability?", seed: "How quickly should we patch a Critical severity vulnerability?"}
  - {id: 30, category: similar_different, expected: miss, query: "What is the difference between a CVE and a GHSA advisory?", seed: "What is the difference between a CVE and a CWE?"}
  - {id: 31, category: unrelated, expected: miss, query: "What is the capital of Australia?"}
  - {id: 32, category: unrelated, expected: miss, query: "How do I bake sourdough bread?"}
  - {id: 33, category: unrelated, expected: miss, query: "Who won the 2018 FIFA World Cup?"}
  - {id: 34, category: unrelated, expected: miss, query: "What is the boiling point of water at sea level?"}
  - {id: 35, category: unrelated, expected: miss, query: "How do I make a pivot table in Excel?"}
  - {id: 36, category: unrelated, expected: miss, query: "What is the tallest mountain in the world?"}
  - {id: 37, category: unrelated, expected: miss, query: "How many planets are in our solar system?"}
  - {id: 38, category: unrelated, expected: miss, query: "What is a good stretching routine before running?"}
  - {id: 39, category: unrelated, expected: miss, query: "How do I change a flat bicycle tire?"}
  - {id: 40, category: unrelated, expected: miss, query: "What is the plot of Romeo and Juliet?"}
```

3.2 Threshold sweep

Threshold sweep code



```python
def load_cases():
    with open(os.path.join(HERE, "cache_cases.yaml")) as f:
        data = yaml.safe_load(f)
    return data["seed_queries"], data["cases"]

def get_agent_answers(questions):
    os.makedirs(RAW_DIR, exist_ok=True)
    answers = {}
    if os.path.exists(ANSWERS_FILE):
        with open(ANSWERS_FILE) as f:
            answers = json.load(f)
    for number, question in enumerate(questions, start=1):
        if question in answers:
            continue
        log(f"Agent run {number} of {len(questions)}: {question}")
        start = time.perf_counter()
        result = asyncio.run(ask_agents(question))
        agent_ms = (time.perf_counter() - start) * 1000
        answers[question] = {"answer": result["answer"], "agent_ms": round(agent_ms, 1)}
        #save after every answer so nothing is lost if the run stops
        with open(ANSWERS_FILE, "w") as f:
            json.dump(answers, f, indent=2)
    return answers

def main():
    seeds, cases = load_cases()
    questions = []
    for question in seeds + [case["query"] for case in cases]:
        if question not in questions:
            questions.append(question)
    answers = get_agent_answers(questions)

    #10 seed answers in an empty cache
    cache = SemanticResponseCache()
    cache.clear()
    for seed in seeds:
        cache.store(seed, cache.embed(seed), answers[seed]["answer"])
    log(f"Stored {len(seeds)} seed answers in the cache")

    #run the 40 cases for each threshold
    rows = []
    false_hits_list = []
    for threshold in THRESHOLDS:
        hits = 0
        false_hits = 0
        latencies = []
        for case in cases:
            start = time.perf_counter()
            best, similarity = cache.find_most_similar(cache.embed(case["query"]))
            lookup_ms = (time.perf_counter() - start) * 1000
            if best is not None and similarity >= threshold:
                hits += 1
                latencies.append(lookup_ms)
                correct = case["expected"] == "hit" and best.query == case.get("seed")
                if not correct:
                    false_hits += 1
                    false_hits_list.append({
                        "threshold": threshold,
                        "case_id": case["id"],
                        "category": case["category"],
                        "incoming_query": case["query"],
                        "cached_query": best.query,
                        "similarity": round(similarity, 3),
                        "cached_answer_returned": best.answer,
                        "correct_answer": answers[case["query"]]["answer"],
                    })
            else:
                latencies.append(lookup_ms + answers[case["query"]]["agent_ms"])

        misses = len(cases) - hits
        rows.append({
            "threshold": threshold,
            "hits": hits,
            "false_hits": false_hits,
            "hit_rate_%": round(hits / len(cases) * 100, 1),
            "false_hit_rate_%": round(false_hits / len(cases) * 100, 1),
            "mean_latency_ms": round(sum(latencies) / len(latencies), 1),
            "total_cost": misses * MISS_COST + false_hits * FALSE_HIT_COST,
        })
    print(f"\nCost model: total cost = misses x {MISS_COST} + false hits x {FALSE_HIT_COST}")
    print("\nResults")
    for r in rows:
        print(f"Threshold {r['threshold']}: hits {r['hits']}, false hits {r['false_hits']}, "
              f"hit rate {r['hit_rate_%']}%, false-hit rate {r['false_hit_rate_%']}%, "
              f"mean latency {r['mean_latency_ms']} ms, total cost {r['total_cost']}")
    best_row = min(rows, key=lambda r: (r["total_cost"], -r["threshold"]))
    print(f"\nLowest total cost: threshold {best_row['threshold']} (cost {best_row['total_cost']})")

    with open(os.path.join(RAW_DIR, "threshold_sweep.csv"), "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
    with open(os.path.join(RAW_DIR, "part3_false_hits.json"), "w") as f:
        json.dump(false_hits_list, f, indent=2)

    #one false hit
    for threshold in reversed(THRESHOLDS):
        at_threshold = [fh for fh in false_hits_list if fh["threshold"] == threshold]
        if at_threshold:
            print(f"\nOne false hit in full (threshold {threshold})")
            false_hit = at_threshold[0]
            print(f"Incoming query: {false_hit['incoming_query']}")
            print(f"Cached query it matched: {false_hit['cached_query']}")
            print(f"Similarity: {false_hit['similarity']}")
            print(f"Cached answer returned: {false_hit['cached_answer_returned']}")
            print(f"Correct answer: {false_hit['correct_answer']}")
            break
    else:
        print("\nNo false hits at any threshold.")

    print("\nSaved raw/threshold_sweep.csv, raw/part3_false_hits.json, raw/part3_agent_answers.json")

if __name__ == "__main__":
    main()
```

Output of run 40 cases at thresholds 0.75, 0.80, 0.85, 0.90, and 0.95

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09> python .\threshold_sweep.py
00:32:48.322 Agent run 1 of 40: What should our team do about CVE-2021-23337 in lodash 4.17.20?
00:32:48.343 Sent the question to s0932_inbox (correlation_id 3112eb1f-5e9e-43c2-89ec-3e4a1fcdcc1d)
00:32:51.458 Got the answer from s0932_final (correlation_id 3112eb1f-5e9e-43c2-89ec-3e4a1fcdcc1d, status approved)
00:32:51.460 Agent run 2 of 40: Which version of requests fixes CVE-2018-18074?
00:32:51.478 Sent the question to s0932_inbox (correlation_id 6a28e710-02d4-4367-a888-09f692d5c12d)
00:32:55.111 Got the answer from s0932_final (correlation_id 6a28e710-02d4-4367-a888-09f692d5c12d, status approved)
00:32:55.113 Agent run 3 of 40: What is the CVSS severity rating for a score of 9.8?
00:32:55.128 Sent the question to s0932_inbox (correlation_id 2224b266-721a-4495-8344-a6c327484b52)
00:32:56.843 Got the answer from s0932_final (correlation_id 2224b266-721a-4495-8344-a6c327484b52, status approved)
00:32:56.845 Agent run 4 of 40: What is a software bill of materials (SBOM)?
00:32:56.858 Sent the question to s0932_inbox (correlation_id 2f57162c-5d93-40fd-bf7e-cdcd1853cbe3)
00:32:58.972 Got the answer from s0932_final (correlation_id 2f57162c-5d93-40fd-bf7e-cdcd1853cbe3, status approved)
00:32:58.974 Agent run 5 of 40: Which log4j version fixes the Log4Shell vulnerability CVE-2021-44228?
00:32:58.988 Sent the question to s0932_inbox (correlation_id 6d8a9dde-d6fa-40f3-bf4a-054d0e4dadcb)
00:33:01.453 Got the answer from s0932_final (correlation_id 6d8a9dde-d6fa-40f3-bf4a-054d0e4dadcb, status approved)
00:33:01.455 Agent run 6 of 40: What is a transitive dependency vulnerability?
00:33:01.470 Sent the question to s0932_inbox (correlation_id c244d3ba-2981-4ad4-a0b3-319a845d4bed)
00:33:03.780 Got the answer from s0932_final (correlation_id c244d3ba-2981-4ad4-a0b3-319a845d4bed, status approved)
00:33:03.781 Agent run 7 of 40: How do we check if our npm project uses a vulnerable package?
00:33:03.795 Sent the question to s0932_inbox (correlation_id c36859d9-c830-4ff5-9494-2609e26ae128)
00:33:06.040 Got the answer from s0932_final (correlation_id c36859d9-c830-4ff5-9494-2609e26ae128, status approved)
00:33:06.051 Agent run 8 of 40: What type of vulnerability is prototype pollution?
00:33:06.064 Sent the question to s0932_inbox (correlation_id 234283cf-1048-43d1-814b-9dfd0807235e)
00:33:08.450 Got the answer from s0932_final (correlation_id 234283cf-1048-43d1-814b-9dfd0807235e, status approved)
00:33:08.452 Agent run 9 of 40: How quickly should we patch a Critical severity vulnerability?
00:33:08.475 Sent the question to s0932_inbox (correlation_id 5fad987a-c6a1-469f-af49-0efd993b9782)
00:33:10.619 Got the answer from s0932_final (correlation_id 5fad987a-c6a1-469f-af49-0efd993b9782, status approved)
00:33:10.621 Agent run 10 of 40: What is the difference between a CVE and a CWE?
00:33:10.635 Sent the question to s0932_inbox (correlation_id db08d31c-caed-4c57-8c09-e6a47df20980)
00:33:13.211 Got the answer from s0932_final (correlation_id db08d31c-caed-4c57-8c09-e6a47df20980, status approved)
00:33:13.212 Agent run 11 of 40: How should we respond to CVE-2021-23337 affecting lodash 4.17.20?
00:33:13.226 Sent the question to s0932_inbox (correlation_id ea0d8ccb-72cd-44c9-b187-bb2edbd88c8b)
00:33:15.762 Got the answer from s0932_final (correlation_id ea0d8ccb-72cd-44c9-b187-bb2edbd88c8b, status approved)
00:33:15.764 Agent run 12 of 40: What requests release patches CVE-2018-18074?
00:33:15.776 Sent the question to s0932_inbox (correlation_id d4f62c7d-64fc-4196-b463-e7044c747ad7)
00:33:18.605 Got the answer from s0932_final (correlation_id d4f62c7d-64fc-4196-b463-e7044c747ad7, status approved)
00:33:18.457 Agent run 13 of 40: Which severity level does a CVSS score of 9.8 fall into?
00:33:18.628 Sent the question to s0932_inbox (correlation_id bb1a9916-0bab-4c7f-ae09-37c03a0652f5)
00:33:20.652 Got the answer from s0932_final (correlation_id bb1a9916-0bab-4c7f-ae09-37c03a0652f5, status approved)
00:33:20.654 Agent run 14 of 40: Can you explain what an SBOM is?
00:33:20.667 Sent the question to s0932_inbox (correlation_id 559aaa0a-86fe-4319-8105-26bd6b0fab1b)
00:33:23.095 Got the answer from s0932_final (correlation_id 559aaa0a-86fe-4319-8105-26bd6b0fab1b, status approved)
00:33:23.097 Agent run 15 of 40: What log4j version should we upgrade to for Log4Shell (CVE-2021-44228)?
00:33:23.110 Sent the question to s0932_inbox (correlation_id afe7c89d-6463-448a-9f98-50d161f22153)
00:33:25.641 Got the answer from s0932_final (correlation_id afe7c89d-6463-448a-9f98-50d161f22153, status approved)
00:33:25.643 Agent run 16 of 40: What does it mean when a vulnerability is in a transitive dependency?
00:33:25.655 Sent the question to s0932_inbox (correlation_id 76b737d4-f9cb-4617-bcc0-85980bc41528)
00:33:28.802 Got the answer from s0932_final (correlation_id 76b737d4-f9cb-4617-bcc0-85980bc41528, status approved)
00:33:28.804 Agent run 17 of 40: How can we find vulnerable packages in our npm project?
00:33:28.818 Sent the question to s0932_inbox (correlation_id 7097454a-3dc8-40f4-9176-e7b3a70b458d)
00:33:30.077 Got the answer from s0932_final (correlation_id 7097454a-3dc8-40f4-9176-e7b3a70b458d, status approved)
00:33:30.079 Agent run 18 of 40: Can you explain the prototype pollution vulnerability type?
00:33:30.092 Sent the question to s0932_inbox (correlation_id f677a965-feee-41bb-977e-98d21953dbea)
00:33:31.880 Got the answer from s0932_final (correlation_id f677a965-feee-41bb-977e-98d21953dbea, status approved)
00:33:32.886 Agent run 19 of 40: What is the recommended time to fix a Critical vulnerability?
00:33:32.898 Sent the question to s0932_inbox (correlation_id 50d0aac1-d392-4f78-a63f-40ad81fbdb95)
00:33:35.336 Got the answer from s0932_final (correlation_id 50d0aac1-d392-4f78-a63f-40ad81fbdb95, status approved)
00:33:35.338 Agent run 20 of 40: How is a CVE different from a CWE?
00:33:35.351 Sent the question to s0932_inbox (correlation_id 72d105fb-0f2a-4e43-9437-bea1153d999e)
00:33:37.758 Got the answer from s0932_final (correlation_id 72d105fb-0f2a-4e43-9437-bea1153d999e, status approved)
00:33:37.760 Agent run 21 of 40: What should our team do about CVE-2020-8203 in lodash 4.17.15?
00:33:37.774 Sent the question to s0932_inbox (correlation_id e80de4d7-5736-48c3-925a-cba131e82a24)
00:33:40.666 Got the answer from s0932_final (correlation_id e80de4d7-5736-48c3-925a-cba131e82a24, status approved)
00:33:40.668 Agent run 22 of 40: Which version of requests fixes CVE-2023-32681?
00:33:40.680 Sent the question to s0932_inbox (correlation_id 971fee04-df53-4598-8113-3eb171baf7ee)
00:33:43.211 Got the answer from s0932_final (correlation_id 971fee04-df53-4598-8113-3eb171baf7ee, status approved)
00:33:43.213 Agent run 23 of 40: What is the CVSS severity rating for a score of 5.3?
00:33:43.226 Sent the question to s0932_inbox (correlation_id 246a6ece-ac99-4242-95c3-b1b26bdd0e35)
00:33:45.551 Got the answer from s0932_final (correlation_id 246a6ece-ac99-4242-95c3-b1b26bdd0e35, status approved)
00:33:45.553 Agent run 24 of 40: What is a vulnerability exploitability exchange (VEX) document?
00:33:45.567 Sent the question to s0932_inbox (correlation_id 11230897-b4e1-465e-9b45-1d25d4a6b37d)
```

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09> python .\threshold_sweep.py
00:33:47.805 Got the answer from s0932_final (correlation_id 11230897-b4e1-465e-9b45-1d25d4a6b37d, status approved)
00:33:47.807 Agent run 25 of 40: Which log4j version fixes CVE-2021-45046?
00:33:47.821 Sent the question to s0932_inbox (correlation_id fb6a86c7-a624-4110-a984-3c948fd9cc92)
00:33:50.998 Got the answer from s0932_final (correlation_id fb6a86c7-a624-4110-a984-3c948fd9cc92, status approved)
00:33:51.000 Agent run 26 of 40: What is a direct dependency vulnerability?
00:33:51.012 Sent the question to s0932_inbox (correlation_id 752299a3-2908-462c-a544-89197d7c74eb)
00:33:53.245 Got the answer from s0932_final (correlation_id 752299a3-2908-462c-a544-89197d7c74eb, status approved)
00:33:53.247 Agent run 27 of 40: How do we check if our Python project uses a vulnerable package?
00:33:53.259 Sent the question to s0932_inbox (correlation_id f2d73177-fa25-4134-9e30-86ff44697203)
00:33:55.621 Got the answer from s0932_final (correlation_id f2d73177-fa25-4134-9e30-86ff44697203, status approved)
00:33:55.624 Agent run 28 of 40: What type of vulnerability is a regular expression denial of service (ReDoS)?
00:33:55.644 Sent the question to s0932_inbox (correlation_id e8bc5a6a-8687-45ff-ac3a-fa0a4edebb15)
00:33:58.107 Got the answer from s0932_final (correlation_id e8bc5a6a-8687-45ff-ac3a-fa0a4edebb15, status approved)
00:33:58.189 Agent run 29 of 40: How quickly should we patch a Low severity vulnerability?
00:33:58.202 Sent the question to s0932_inbox (correlation_id 55ac5029-f537-4f60-86fc-2c4df40f0218)
00:34:00.550 Got the answer from s0932_final (correlation_id 55ac5029-f537-4f60-86fc-2c4df40f0218, status approved)
00:34:00.552 Agent run 30 of 40: What is the difference between a CVE and a GHSA advisory?
00:34:00.567 Sent the question to s0932_inbox (correlation_id 1b955eab-64c8-40fb-8cb1-d03a9cb1e35f)
00:34:03.370 Got the answer from s0932_final (correlation_id 1b955eab-64c8-40fb-8cb1-d03a9cb1e35f, status approved)
00:34:03.373 Agent run 31 of 40: What is the capital of Australia?
00:34:03.385 Sent the question to s0932_inbox (correlation_id f8394856-9a19-4aea-9c05-e33356078787)
00:34:04.442 Got the answer from s0932_final (correlation_id f8394856-9a19-4aea-9c05-e33356078787, status approved)
00:34:04.445 Agent run 32 of 40: How do I bake sourdough bread?
00:34:04.458 Sent the question to s0932_inbox (correlation_id 2983532a-3095-40a0-b421-35213ddd1d9b)
00:34:06.837 Got the answer from s0932_final (correlation_id 2983532a-3095-40a0-b421-35213ddd1d9b, status approved)
00:34:06.840 Agent run 33 of 40: Who won the 2018 FIFA World Cup?
00:34:06.853 Sent the question to s0932_inbox (correlation_id 2154a198-36f0-45e7-8ec9-0a1639e3a7b0)
00:34:08.257 Got the answer from s0932_final (correlation_id 2154a198-36f0-45e7-8ec9-0a1639e3a7b0, status approved)
00:34:08.259 Agent run 34 of 40: What is the boiling point of water at sea level?
00:34:08.273 Sent the question to s0932_inbox (correlation_id 52ae244f-7cda-44d8-adf9-e46bd7dbd5a2)
00:34:09.996 Got the answer from s0932_final (correlation_id 52ae244f-7cda-44d8-adf9-e46bd7dbd5a2, status approved)
00:34:09.999 Agent run 35 of 40: How do I make a pivot table in Excel?
00:34:10.010 Sent the question to s0932_inbox (correlation_id 0314aba7-5a8c-458a-a49f-7bfff8e106a7)
00:34:12.071 Got the answer from s0932_final (correlation_id 0314aba7-5a8c-458a-a49f-7bfff8e106a7, status approved)
00:34:12.074 Agent run 36 of 40: What is the tallest mountain in the world?
00:34:12.087 Sent the question to s0932_inbox (correlation_id 92401820-868a-453c-a917-3b854e8a3c92)
00:34:14.170 Got the answer from s0932_final (correlation_id 92401820-868a-453c-a917-3b854e8a3c92, status approved)
00:34:14.172 Agent run 37 of 40: How many planets are in our solar system?
00:34:14.185 Sent the question to s0932_inbox (correlation_id bf81d398-092e-4c5e-a9a6-9c19085e7a1e)
00:34:16.443 Got the answer from s0932_final (correlation_id bf81d398-092e-4c5e-a9a6-9c19085e7a1e, status approved)
00:34:16.445 Agent run 38 of 40: What is a good stretching routine before running?
00:34:16.458 Sent the question to s0932_inbox (correlation_id 3ad7c55f-4c26-45cd-8941-f6047722287ff)
00:34:18.943 Got the answer from s0932_final (correlation_id 3ad7c55f-4c26-45cd-8941-f6047722287ff, status approved)
00:34:18.945 Agent run 39 of 40: How do I change a flat bicycle tire?
00:34:18.958 Sent the question to s0932_inbox (correlation_id 328e4bd5-f93e-4c27-9ef9-4b12043abbbe)
00:34:21.196 Got the answer from s0932_final (correlation_id 328e4bd5-f93e-4c27-9ef9-4b12043abbbe, status approved)
00:34:21.198 Agent run 40 of 40: What is the plot of Romeo and Juliet?
00:34:21.218 Sent the question to s0932_inbox (correlation_id 2f846008-9334-4bf6-b94d-7bc48d75404e)
00:34:23.798 Got the answer from s0932_final (correlation_id 2f846008-9334-4bf6-b94d-7bc48d75404e, status approved)
Loading weights: 100%|▌| 103/103 [00:00<00:00, 4732.68it/s]
00:34:24.435 Created vector index s0932:semantic_cache_idx
00:34:24.551 Stored 10 seed answers in the cache
```

Agents outputs for planner

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\planner.py
00:30:57.231 PLANNER Listening on s0932_inbox…
00:30:57.232 PLANNER RECEIVE from s0932_inbox partition=0 offset=8 correlation_id=30da83a4-f7c2-4eec-be26-d0ba9d63cdc6
00:30:57.232 PLANNER question=What should our team do about CVE-2021-23337 in lodash 4.17.20?
00:31:02.430 PLANNER SEND to s0932_tasks partition=0 offset=6 correlation_id=30da83a4-f7c2-4eec-be26-d0ba9d63cdc6
00:31:02.430 PLANNER plan=1. Verify if the affected version of lodash (4.17.20) is actually in use by your project.
2. Determine the severity of the vulnerability and whether it impacts your specific use case.
3. Plan to update to a patched version of lodash or apply a mitigation strategy if an update is not immediately feasible.
00:32:48.343 PLANNER RECEIVE from s0932_inbox partition=0 offset=9 correlation_id=3112eb1f-5e9e-43c2-89ec-3e4a1fcdcc1d
00:32:48.343 PLANNER question=What should our team do about CVE-2021-23337 in lodash 4.17.20?
00:32:49.558 PLANNER SEND to s0932_tasks partition=1 offset=2 correlation_id=3112eb1f-5e9e-43c2-89ec-3e4a1fcdcc1d
00:32:49.558 PLANNER plan=1. Verify if the affected version of lodash (4.17.20) is actually in use by your project.
2. Determine the severity of the vulnerability and whether it impacts your specific use case.
3. Plan to update to a patched version of lodash or apply a mitigation strategy if an update is not immediately feasible.
00:32:51.478 PLANNER RECEIVE from s0932_inbox partition=0 offset=10 correlation_id=6a28e710-02d4-4367-a888-09f692d5c12d
00:32:51.478 PLANNER question=Which version of requests fixes CVE-2018-18074?
00:32:52.427 PLANNER SEND to s0932_tasks partition=0 offset=7 correlation_id=6a28e710-02d4-4367-a888-09f692d5c12d
00:32:52.427 PLANNER plan=1. Identify the specific CVE-2018-18074 vulnerability description to understand the issue.
2. Check the release notes or security advisories of the `requests` package to find the version that includes the fix.
3. Verify the fix by testing the specified version or referencing a trusted security database like CVE details or PyPI.
00:32:55.128 PLANNER RECEIVE from s0932_inbox partition=0 offset=11 correlation_id=2224b266-721a-4495-8344-a6c327484b52
00:32:55.128 PLANNER question=What is the CVSS severity rating for a score of 9.8?
00:32:55.866 PLANNER SEND to s0932_tasks partition=0 offset=8 correlation_id=2224b266-721a-4495-8344-a6c327484b52
00:32:55.866 PLANNER plan=1. Identify the CVSS version being referenced (CVSS v2 or CVSS v3).
2. Determine the severity rating based on the score range for that CVSS version.
3. Provide the corresponding severity level for a CVSS score of 9.8.
00:32:56.859 PLANNER RECEIVE from s0932_inbox partition=0 offset=12 correlation_id=2f57162c-5d93-40fd-bf7e-cdcd1853cbe3
00:32:56.859 PLANNER question=What is a software bill of materials (SBOM)?
00:32:57.421 PLANNER SEND to s0932_tasks partition=1 offset=3 correlation_id=2f57162c-5d93-40fd-bf7e-cdcd1853cbe3
00:32:57.421 PLANNER plan=1. Define what an SBOM is and its purpose in software development.
2. Explain the typical components included in an SBOM.
3. Describe the benefits of using an SBOM for vulnerability management.
00:32:58.988 PLANNER RECEIVE from s0932_inbox partition=0 offset=13 correlation_id=6d8a9dde-d6fa-40f3-bf4a-054d0e4dadcb
00:32:58.988 PLANNER question=Which log4j version fixes the Log4Shell vulnerability CVE-2021-44228?
00:32:59.813 PLANNER SEND to s0932_tasks partition=1 offset=4 correlation_id=6d8a9dde-d6fa-40f3-bf4a-054d0e4dadcb
00:32:59.813 PLANNER plan=1. Identify the specific Log4j version associated with the Log4Shell vulnerability (CVE-2021-44228).
2. Determine the minimum version that provides a patch for the vulnerability.
3. Verify the recommended version from the official Log4j release notes or security advisories.
00:33:01.471 PLANNER RECEIVE from s0932_inbox partition=0 offset=14 correlation_id=c244d3ba-2981-4ad4-a0b3-319a845d4bed
00:33:01.471 PLANNER question=What is a transitive dependency vulnerability?
00:33:02.062 PLANNER SEND to s0932_tasks partition=1 offset=5 correlation_id=c244d3ba-2981-4ad4-a0b3-319a845d4bed
00:33:02.062 PLANNER plan=1. Define what a transitive dependency is in the context of software development.
2. Explain how a transitive dependency vulnerability occurs.
3. Describe the potential risks and impacts of such vulnerabilities.
00:33:03.795 PLANNER RECEIVE from s0932_inbox partition=0 offset=15 correlation_id=c36859d9-c830-4ff5-9494-2609e26ae128
00:33:03.795 PLANNER question=How do we check if our npm project uses a vulnerable package?
00:33:04.541 PLANNER SEND to s0932_tasks partition=0 offset=9 correlation_id=c36859d9-c830-4ff5-9494-2609e26ae128
00:33:04.541 PLANNER plan=1. Run `npm audit` to identify any known vulnerabilities in your project's dependencies.
2. Review the audit report to see which packages are flagged as vulnerable.
3. Check the specific versions of the vulnerable packages and determine if they are currently in use in your project.
00:33:06.065 PLANNER RECEIVE from s0932_inbox partition=0 offset=16 correlation_id=234283cf-1048-43d1-814b-9dfd0807235e
00:33:06.065 PLANNER question=What type of vulnerability is prototype pollution?
00:33:06.570 PLANNER SEND to s0932_tasks partition=1 offset=6 correlation_id=234283cf-1048-43d1-814b-9dfd0807235e
00:33:06.570 PLANNER plan=1. Define prototype pollution and explain how it occurs.
2. Describe the potential impact of prototype pollution on application security.
3. Provide examples of how prototype pollution can be exploited.
00:33:08.475 PLANNER RECEIVE from s0932_inbox partition=0 offset=17 correlation_id=5fad987a-c6a1-469f-af49-0efd993b9782
00:33:08.475 PLANNER question=How quickly should we patch a Critical severity vulnerability?
00:33:09.434 PLANNER SEND to s0932_tasks partition=0 offset=10 correlation_id=5fad987a-c6a1-469f-af49-0efd993b9782
00:33:09.434 PLANNER plan=1. Assess the immediate risk to your systems and users based on the vulnerability's impact and potential exploitability.
2. Prioritize patching based on the criticality of affected systems, user impact, and availability of a reliable fix.
3. Implement the patch as soon as possible, ideally within 24-48 hours, and communicate the action to stakeholders.
00:33:10.635 PLANNER RECEIVE from s0932_inbox partition=0 offset=18 correlation_id=db08d31c-caed-4c57-8c09-e6a47df20980
00:33:10.636 PLANNER question=What is the difference between a CVE and a CWE?
```

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\planner.py
00:33:11.332 PLANNER SEND to s0932_tasks partition=1 offset=7 correlation_id=db08d31c-caed-4c57-8c09-e6a47df20980
00:33:11.332 PLANNER plan=1. Define what a CVE is and its purpose in identifying specific vulnerabilities.
2. Explain what a CWE is and its role in categorizing types of software weaknesses.
3. Highlight the key difference between CVEs and CWEs in terms of their focus and usage.
00:33:13.227 PLANNER RECEIVE from s0932_inbox partition=0 offset=19 correlation_id=ea0d8ccb-72cd-44c9-b187-bb2edbd88c8b
00:33:13.227 PLANNER question=How should we respond to CVE-2021-23337 affecting lodash 4.17.20?
00:33:14.067 PLANNER SEND to s0932_tasks partition=0 offset=11 correlation_id=ea0d8ccb-72cd-44c9-b187-bb2edbd88c8b
00:33:14.067 PLANNER plan=1. Verify the impact of CVE-2021-23337 on your current usage of lodash 4.17.20.
2. Update lodash to a patched version immediately to mitigate the vulnerability.
3. Monitor for any related issues and communicate the update to your users or stakeholders.
00:33:15.777 PLANNER RECEIVE from s0932_inbox partition=0 offset=20 correlation_id=d4f62c7d-64fc-4196-b463-e7044c747ad7
00:33:15.777 PLANNER question=What requests release patches CVE-2018-18074?
00:33:16.722 PLANNER SEND to s0932_tasks partition=0 offset=12 correlation_id=d4f62c7d-64fc-4196-b463-e7044c747ad7
00:33:16.722 PLANNER plan=1. Identify the specific software or package associated with the CVE-2018-18074 vulnerability.
2. Determine which organizations or maintainers are responsible for releasing patches for that vulnerability.
3. Research and list the official requests or reports that prompted the release of the patch for CVE-2018-18074.
00:33:18.628 PLANNER RECEIVE from s0932_inbox partition=0 offset=21 correlation_id=bb1a9916-0bab-4c7f-ae09-37c03a0652f5
00:33:18.628 PLANNER question=Which severity level does a CVSS score of 9.8 fall into?
00:33:19.250 PLANNER SEND to s0932_tasks partition=1 offset=8 correlation_id=bb1a9916-0bab-4c7f-ae09-37c03a0652f5
00:33:19.250 PLANNER plan=1. Identify the CVSS score range for each severity level.
2. Determine which severity level corresponds to a CVSS score of 9.8.
3. Provide the correct severity level based on the CVSS scoring criteria.
00:33:20.667 PLANNER RECEIVE from s0932_inbox partition=0 offset=22 correlation_id=559aaa0a-86fe-4319-8105-26bd6b0fab1b
00:33:20.667 PLANNER question=Can you explain what an SBOM is?
00:33:21.386 PLANNER SEND to s0932_tasks partition=1 offset=9 correlation_id=559aaa0a-86fe-4319-8105-26bd6b0fab1b
00:33:21.386 PLANNER plan=1. Define what an SBOM is and its purpose in software development.
2. Explain the common formats used for SBOMs, such as SPDX, CycloneDX, and SWID.
3. Describe how SBOMs are used to identify and manage software vulnerabilities.
00:33:23.110 PLANNER RECEIVE from s0932_inbox partition=0 offset=23 correlation_id=afe7c89d-6463-448a-9f98-50d161f22153
00:33:23.110 PLANNER question=What log4j version should we upgrade to for Log4Shell (CVE-2021-44228)?
00:33:23.202 PLANNER SEND to s0932_tasks partition=1 offset=10 correlation_id=afe7c89d-6463-448a-9f98-50d161f22153
00:33:23.869 PLANNER plan=1. Identify the affected Log4j versions for Log4Shell (CVE-2021-44228).
2. Check the official Log4j release notes for the latest secure version.
3. Upgrade to the recommended version and verify the fix is applied.
00:33:25.655 PLANNER RECEIVE from s0932_inbox partition=0 offset=24 correlation_id=76b737d4-f9cb-4617-bcc0-85980bc41528
00:33:25.655 PLANNER question=What does it mean when a vulnerability is in a transitive dependency?
00:33:26.360 PLANNER SEND to s0932_tasks partition=1 offset=11 correlation_id=76b737d4-f9cb-4617-bcc0-85980bc41528
00:33:26.360 PLANNER plan=1. Identify the main package and the transitive dependency chain.
2. Explain how the vulnerability exists in a dependency that is not directly listed in the main package's dependencies.
3. Describe the potential impact and how to address the vulnerability in the transitive dependency.
00:33:28.018 PLANNER RECEIVE from s0932_inbox partition=0 offset=25 correlation_id=7097454a-3dc8-40f4-9176-e7b3a70b458d
00:33:28.018 PLANNER question=How can we find vulnerable packages in our npm project?
00:33:28.686 PLANNER SEND to s0932_tasks partition=0 offset=13 correlation_id=7097454a-3dc8-40f4-9176-e7b3a70b458d
00:33:28.686 PLANNER plan=1. Use `npm audit` to scan your project for known vulnerabilities.
2. Review the audit report to identify affected packages and their versions.
3. Update the vulnerable packages to secure versions using `npm update` or manually edit `package.json`.
00:33:30.092 PLANNER RECEIVE from s0932_inbox partition=0 offset=26 correlation_id=f677a965-feee-41bb-977e-98d21953dbea
00:33:30.092 PLANNER question=Can you explain the prototype pollution vulnerability type?
00:33:30.693 PLANNER SEND to s0932_tasks partition=1 offset=12 correlation_id=f677a965-feee-41bb-977e-98d21953dbea
00:33:30.693 PLANNER plan=1. Define what prototype pollution is and how it occurs in JavaScript objects.
2. Explain the potential security risks and impacts of prototype pollution.
3. Provide examples of how prototype pollution can be exploited and how to prevent it.
00:33:32.898 PLANNER RECEIVE from s0932_inbox partition=0 offset=27 correlation_id=50d0aac1-d392-4f78-a63f-40ad81fbdb95
00:33:32.898 PLANNER question=What is the recommended time to fix a Critical vulnerability?
00:33:33.561 PLANNER SEND to s0932_tasks partition=0 offset=14 correlation_id=50d0aac1-d392-4f78-a63f-40ad81fbdb95
00:33:33.561 PLANNER plan=1. Assess the severity and impact of the critical vulnerability to prioritize the fix.
2. Coordinate with the development team to identify and implement a secure patch or update.
3. Communicate the fix timeline to stakeholders and ensure the patch is deployed promptly.
00:33:35.351 PLANNER RECEIVE from s0932_inbox partition=0 offset=28 correlation_id=72d105fb-0f2a-4e43-9437-bea1153d999e
00:33:35.351 PLANNER question=How is a CVE different from a CWE?
00:33:36.456 PLANNER SEND to s0932_tasks partition=1 offset=13 correlation_id=72d105fb-0f2a-4e43-9437-bea1153d999e
00:33:36.456 PLANNER plan=1. Identify the purpose of CVE and CWE: CVE is a unique identifier for publicly known cybersecurity vulnerabilities, while CWE is a classification system for common weakness types.
2. Compare their focus areas: CVE focuses on specific vulnerabilities in software, whereas CWE focuses on the underlying causes or weaknesses
```



```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\planner.py
 that lead to vulnerabilities.
3. Explain their relationship: CVEs often reference CWEs to provide context on the type of weakness that contributed to the vulnerability.
00:33:37.775 PLANNER RECEIVE from s0932_inbox partition=0 offset=29 correlation_id=e80de4d7-5736-48c3-925a-cba131e82a24
00:33:37.775 PLANNER question=How should our team do about CVE-2020-8203 in lodash 4.17.15?
00:33:38.861 PLANNER SEND to s0932_tasks partition=0 offset=15 correlation_id=e80de4d7-5736-48c3-925a-cba131e82a24
00:33:38.861 PLANNER plan=1. Verify if the affected version of lodash (4.17.15) is still in use by your project or its dependencies.
2. Check if there is an updated version of lodash that addresses CVE-2020-8203 and is compatible with your project.
3. Update the affected dependency to the secure version and ensure all dependent packages are also updated to avoid reintroducing the vulnera
bility.
00:33:40.681 PLANNER RECEIVE from s0932_inbox partition=0 offset=30 correlation_id=071fee04-df53-4598-8113-3eb171baf7ee
00:33:40.681 PLANNER question=Which version of requests fixes CVE-2023-32681?
00:33:41.617 PLANNER SEND to s0932_tasks partition=0 offset=16 correlation_id=071fee04-df53-4598-8113-3eb171baf7ee
00:33:41.617 PLANNER plan=1. Identify the specific CVE-2023-32681 vulnerability details and its impact.
2. Check the official release notes or security advisories of the `requests` package for the affected version.
3. Determine the minimum version of `requests` that includes a fix for CVE-2023-32681.
00:33:43.226 PLANNER RECEIVE from s0932_inbox partition=0 offset=31 correlation_id=246a6ece-ac99-4242-95c3-b1b26bdd0e35
00:33:43.226 PLANNER question=What is the CVSS severity rating for a score of 5.3?
00:33:44.150 PLANNER SEND to s0932_tasks partition=0 offset=17 correlation_id=246a6ece-ac99-4242-95c3-b1b26bdd0e35
00:33:44.150 PLANNER plan=1. Identify the CVSS version being used (e.g., CVSS v3.0 or v3.1).
2. Use the CVSS scoring criteria to determine the severity rating based on the score of 5.3.
3. Provide the corresponding severity level (e.g., Medium, Low, etc.) based on the CVSS scale.
00:33:45.567 PLANNER RECEIVE from s0932_inbox partition=0 offset=32 correlation_id=11230897-b4e1-465e-9b45-1d2504a6b37d
00:33:45.567 PLANNER question=What is a vulnerability exploitability exchange (VEX) document?
00:33:46.246 PLANNER SEND to s0932_tasks partition=1 offset=14 correlation_id=11230897-b4e1-465e-9b45-1d2504a6b37d
00:33:46.247 PLANNER plan=1. Define what a VEX document is and its purpose in the context of software security.
2. Explain the key components and structure of a VEX document.
3. Describe how VEX documents are used by open source package vulnerability teams.
00:33:47.821 PLANNER RECEIVE from s0932_inbox partition=0 offset=33 correlation_id=fb6a86c7-a624-4110-a984-3c948fd9cc92
00:33:47.821 PLANNER question=Which log4j version fixes CVE-2021-45046?
00:33:48.723 PLANNER SEND to s0932_tasks partition=0 offset=18 correlation_id=fb6a86c7-a624-4110-a984-3c948fd9cc92
00:33:48.723 PLANNER plan=1. Identify the specific CVE-2021-45046 vulnerability description to understand the exact issue.
2. Check the official log4j release notes or security advisories to find the version that includes the fix.
3. Confirm the recommended version from reliable sources like the Apache Software Foundation or cybersecurity databases.
00:33:51.013 PLANNER RECEIVE from s0932_inbox partition=0 offset=34 correlation_id=752299a3-2988-462c-a544-89197d7c74eb
00:33:51.013 PLANNER question=what is a direct dependency vulnerability?
00:33:51.669 PLANNER SEND to s0932_tasks partition=0 offset=19 correlation_id=752299a3-2988-462c-a544-89197d7c74eb
00:33:51.669 PLANNER plan=1. Define what a direct dependency is in the context of software development.
2. Explain how a vulnerability in a direct dependency can affect the main project.
3. Provide examples of common tools or methods used to identify and manage direct dependency vulnerabilities.
00:33:53.260 PLANNER RECEIVE from s0932_inbox partition=0 offset=35 correlation_id=f2d73177-fa25-4134-9e30-86ff44697203
00:33:53.260 PLANNER question=How do we check if our Python project uses a vulnerable package?
00:33:54.085 PLANNER SEND to s0932_tasks partition=1 offset=15 correlation_id=f2d73177-fa25-4134-9e30-86ff44697203
00:33:54.085 PLANNER plan=1. Identify all dependencies in your Python project using tools like `pip freeze` or `pipdeptree`.
2. Check each package version against known vulnerabilities using a tool like `safety` or `bandit`.
3. Review the results to determine if any vulnerable packages are in use and update them accordingly.
00:33:55.645 PLANNER RECEIVE from s0932_inbox partition=0 offset=36 correlation_id=e80c5a5e-8687-45ff-ac3a-fa0a4edebb15
00:33:55.645 PLANNER question=what type of vulnerability is a regular expression denial of service (ReDoS)?
00:33:56.401 PLANNER SEND to s0932_tasks partition=1 offset=16 correlation_id=e80c5a5e-8687-45ff-ac3a-fa0a4edebb15
00:33:56.401 PLANNER plan=1. Identify the nature of a regular expression denial of service (ReDoS) vulnerability.
2. Explain how ReDoS exploits poorly crafted regular expressions to cause performance issues.
3. Describe the impact of ReDoS, such as resource exhaustion and service disruption.
00:33:58.202 PLANNER RECEIVE from s0932_inbox partition=0 offset=37 correlation_id=55ac5029-f537-4f60-86fc-2c4df40f0218
00:33:58.202 PLANNER question=How quickly should we patch a low severity vulnerability?
00:33:58.920 PLANNER SEND to s0932_tasks partition=0 offset=20 correlation_id=55ac5029-f537-4f60-86fc-2c4df40f0218
00:33:58.920 PLANNER plan=1. Assess the vulnerability's impact and whether it affects your specific use case.
2. Review your organization's security policies and patching guidelines for severity-based timelines.
3. Prioritize the patch based on risk, system criticality, and available mitigation strategies.
00:34:00.567 PLANNER RECEIVE from s0932_inbox partition=0 offset=38 correlation_id=1b955eab-64c8-40fb-8cb1-d03a9cb1e35f
00:34:00.567 PLANNER question=What is the difference between a CVE and a GHSA advisory?
00:34:01.285 PLANNER SEND to s0932_tasks partition=0 offset=21 correlation_id=1b955eab-64c8-40fb-8cb1-d03a9cb1e35f
00:34:01.285 PLANNER plan=1. Identify the purpose and origin of CVEs and GHSA advisories.
2. Compare how each type of advisory is structured and used in vulnerability management.
3. Highlight the key differences in their scope, format, and usage by the security community.
00:34:03.385 PLANNER RECEIVE from s0932_inbox partition=0 offset=39 correlation_id=f8394856-9a19-4aea-9c05-e33356078787
00:34:03.385 PLANNER question=What is the capital of Australia?
00:34:03.787 PLANNER SEND to s0932_tasks partition=1 offset=17 correlation_id=f8394856-9a19-4aea-9c05-e33356078787
00:34:03.787 PLANNER plan=1. Identify the country in question.
```

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\planner.py
2. Recall or look up the capital city of Australia.
3. Provide the correct answer.
00:34:04.458 PLANNER RECEIVE from s0932_inbox partition=0 offset=40 correlation_id=2983532a-3095-40a0-b421-352136dd1d9b
00:34:04.458 PLANNER question=How do I bake sourdough bread?
00:34:05.226 PLANNER SEND to s0932_tasks partition=1 offset=18 correlation_id=2983532a-3095-40a0-b421-352136dd1d9b
00:34:05.226 PLANNER plan=1. Prepare and maintain a sourdough starter by feeding it with flour and water.
2. Mix flour, water, and a portion of the sourdough starter to form the dough.
3. Let the dough ferment and rise, then shape and bake according to your recipe.
00:34:06.853 PLANNER RECEIVE from s0932_inbox partition=0 offset=41 correlation_id=2154a198-36f0-45e7-8ec9-0a1639e3a7b0
00:34:06.853 PLANNER question=Who won the 2018 FIFA World Cup?
00:34:07.394 PLANNER SEND to s0932_tasks partition=0 offset=22 correlation_id=2154a198-36f0-45e7-8ec9-0a1639e3a7b0
00:34:07.394 PLANNER plan=1. Identify the winning team of the 2018 FIFA World Cup final.
2. Confirm the opponent team that lost the final.
3. State the score of the final match.
00:34:08.274 PLANNER RECEIVE from s0932_inbox partition=0 offset=42 correlation_id=52ae244f-7c8a-44d8-adf9-e46bd7dbd5a2
00:34:08.274 PLANNER question=What is the boiling point of water at sea level?
00:34:08.846 PLANNER SEND to s0932_tasks partition=1 offset=19 correlation_id=52ae244f-7c8a-44d8-adf9-e46bd7dbd5a2
00:34:08.847 PLANNER plan=1. Identify the standard atmospheric pressure at sea level.
2. Recall the temperature at which water reaches its boiling point under that pressure.
3. Provide the boiling point of water in degrees Celsius or Fahrenheit.
00:34:10.020 PLANNER RECEIVE from s0932_inbox partition=0 offset=43 correlation_id=0314a0a7-5a8c-450a-a49f-7bfff8e106a7
00:34:10.020 PLANNER question=How do I make a pivot table in Excel?
00:34:10.873 PLANNER SEND to s0932_tasks partition=1 offset=20 correlation_id=0314a0a7-5a8c-450a-a49f-7bfff8e106a7
00:34:10.873 PLANNER plan=1. Select the data range you want to use for the pivot table.
2. Go to the 'Insert' tab and click 'PivotTable' to create a new pivot table.
3. Choose where to place the pivot table and drag fields to the Rows, Columns, Values, and Filters areas.
00:34:12.087 PLANNER RECEIVE from s0932_inbox partition=0 offset=44 correlation_id=92401820-868a-453c-a917-3b854e8a3c92
00:34:12.087 PLANNER question=What is the tallest mountain in the world?
00:34:12.679 PLANNER SEND to s0932_tasks partition=1 offset=21 correlation_id=92401820-868a-453c-a917-3b854e8a3c92
00:34:12.679 PLANNER plan=1. Identify the mountain that is officially recognized as the tallest in the world.
2. Confirm its height and location based on reliable geographical sources.
3. Provide the name and elevation of the mountain as the answer.
00:34:14.185 PLANNER RECEIVE from s0932_inbox partition=0 offset=45 correlation_id=bf81d398-092e-4c5e-a9a5-9c19805e7a1e
00:34:14.185 PLANNER question=How many planets are in our solar system?
00:34:14.818 PLANNER SEND to s0932_tasks partition=0 offset=23 correlation_id=bf81d398-092e-4c5e-a9a5-9c19805e7a1e
00:34:14.818 PLANNER plan=1. Identify the current scientific consensus on the number of planets in our solar system.
2. List the recognized planets based on their orbital characteristics and classification.
3. Confirm if any recently discovered objects qualify as planets under the current definition.
00:34:16.459 PLANNER RECEIVE from s0932_inbox partition=0 offset=46 correlation_id=3ad7c55f-4c26-45cd-8941-f604772287ff
00:34:16.459 PLANNER question=What is a good stretching routine before running?
00:34:17.225 PLANNER SEND to s0932_tasks partition=0 offset=24 correlation_id=3ad7c55f-4c26-45cd-8941-f604772287ff
00:34:17.225 PLANNER plan=1. Warm up with light cardio for 5-10 minutes.
2. Perform dynamic stretches like leg swings, hip circles, and walking lunges.
3. Focus on major muscle groups used in running, such as quads, hamstrings, calves, and hip flexors.
00:34:18.958 PLANNER RECEIVE from s0932_inbox partition=0 offset=47 correlation_id=328e4bd5-f93e-4c27-9ef9-4b12043abbbe
00:34:18.958 PLANNER question=How do I change a flat bicycle tire?
00:34:19.838 PLANNER SEND to s0932_tasks partition=0 offset=25 correlation_id=328e4bd5-f93e-4c27-9ef9-4b12043abbbe
00:34:19.838 PLANNER plan=1. Remove the wheel from the bicycle by loosening the quick release skewer or bolts.
2. Take off the old tire by deflating the inner tube and carefully peeling the tire over the rim.
3. Insert the new tire onto the rim, inflate the inner tube, and reattach the wheel to the bicycle.
00:34:21.219 PLANNER RECEIVE from s0932_inbox partition=0 offset=48 correlation_id=2f846008-9334-4bf6-b94d-7bc48d75404e
00:34:21.219 PLANNER question=What is the plot of Romeo and Juliet?
00:34:21.763 PLANNER SEND to s0932_tasks partition=0 offset=26 correlation_id=2f846008-9334-4bf6-b94d-7bc48d75404e
00:34:21.764 PLANNER plan=1. Identify the main characters and setting of Romeo and Juliet.
2. Outline the key events that drive the story forward.
3. Summarize the central conflict and resolution of the play.
```

Agents outputs for writer and reviewer

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\writer.py
00:31:20.511 [WRITER-A] Listening on s0932_tasks... mode=AFTER FIX (idempotency, commit after success, 3 retries + dead letter)
00:31:20.512 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=6
00:31:20.512 [WRITER-A] correlation_id=30da83a4-f7c2-4eec-be26-d0ba9d63cdc6
00:31:22.045 [WRITER-A] SEND to s0932_drafts partition=0 offset=8 correlation_id=30da83a4-f7c2-4eec-be26-d0ba9d63cdc6
00:31:22.045 [WRITER-A] draft=Verify if your project uses lodash 4.17.20. If so, assess the vulnerability's impact on your code. CVE-2021-233
37 allows remote code execution, making it high severity. Update to a patched version (e.g., 4.17.21 or later) as soon as possible. If an upd
ate isn't feasible, implement input validation or isolate sensitive operations to mitigate risk. Prioritize patching to ensure security.
00:31:22.047 [WRITER-A] COMMIT partition=0 offset=6 (next offset to read: 7)
00:32:49.550 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=2
00:32:49.559 [WRITER-A] correlation_id=3112eb1f-5e9e-43c2-89ec-3e4a1fcdcc1d
00:32:50.821 [WRITER-A] SEND to s0932_drafts partition=0 offset=9 correlation_id=3112eb1f-5e9e-43c2-89ec-3e4a1fcdcc1d
00:32:50.821 [WRITER-A] draft=Verify if your project uses lodash 4.17.20. If so, assess the vulnerability's impact on your code. CVE-2021-233
37 allows remote code execution, making it critical. Update to a patched version (e.g., 4.17.21 or later) as soon as possible. If an update i
sn't feasible, implement a mitigation strategy, such as input sanitization or limiting lodash usage. Prioritize security to prevent potential
 exploitation.
00:32:50.823 [WRITER-A] COMMIT partition=1 offset=2 (next offset to read: 3)
00:32:52.427 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=7
00:32:52.428 [WRITER-A] correlation_id=6a28e710-02d4-4367-a888-09f692d5c12d
00:32:54.569 [WRITER-A] SEND to s0932_drafts partition=0 offset=10 correlation_id=6a28e710-02d4-4367-a888-09f692d5c12d
00:32:54.569 [WRITER-A] draft=CVE-2018-18074 relates to a vulnerability in the `requests` package where improper handling of certain HTTP hea
ders could lead to information disclosure. The fix for this vulnerability was included in `requests` version 2.19.1, released on April 10, 20
19. This version addresses the issue by properly validating and sanitizing headers. You can verify this fix by checking the [PyPI release not
es](https://pypi.org/project/requests/2.19.1/) or the [CVE details page](https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-18074). Upgr
ading to version 2.19.1 or later is recommended to resolve this vulnerability.
00:32:54.571 [WRITER-A] COMMIT partition=0 offset=7 (next offset to read: 8)
00:32:55.867 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=8
00:32:55.867 [WRITER-A] correlation_id=2224b266-721a-4495-8344-a6c327484b52
00:32:56.493 [WRITER-A] SEND to s0932_drafts partition=0 offset=11 correlation_id=2224b266-721a-4495-8344-a6c327484b52
00:32:56.493 [WRITER-A] draft=CVSS v3 is used for scores above 7.0. A CVSS score of 9.8 falls within the range of 9.0-10.0, which corresponds
 to a **Critical** severity rating.
00:32:56.495 [WRITER-A] COMMIT partition=0 offset=8 (next offset to read: 9)
00:32:57.422 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=3
00:32:57.422 [WRITER-A] correlation_id=2f57162c-5d93-40fd-bf7e-cdcd1853cbe3
00:32:58.591 [WRITER-A] SEND to s0932_drafts partition=0 offset=12 correlation_id=2f57162c-5d93-40fd-bf7e-cdcd1853cbe3
00:32:58.591 [WRITER-A] draft=An SBOM is a formal document listing all the components, libraries, and dependencies in a software product. Its
 purpose is to provide transparency and visibility into the software supply chain. Typical components include software names, versions, licen
ses, and cryptographic hashes. Using an SBOM enhances vulnerability management by enabling quick identification and remediation of security i
ssues, ensuring compliance, and supporting incident response. It also aids in tracking dependencies and reducing the risk of using outdated o
r insecure components.
00:32:58.592 [WRITER-A] COMMIT partition=1 offset=3 (next offset to read: 4)
00:32:59.814 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=4
00:32:59.814 [WRITER-A] correlation_id=6d8a9dde-d6fa-40f3-bf4a-054d0e4dadcb
00:33:00.976 [WRITER-A] SEND to s0932_drafts partition=0 offset=13 correlation_id=6d8a9dde-d6fa-40f3-bf4a-054d0e4dadcb
00:33:00.976 [WRITER-A] draft=The Log4Shell vulnerability (CVE-2021-44228) is fixed in Log4j version 2.15.0 and later. The minimum recommende
d version to address the vulnerability is 2.15.0, as confirmed by the Apache Software Foundation in their official release notes and security
 advisories. Upgrading to this version or newer ensures the patch is applied, mitigating the risk of remote code execution.
00:33:00.978 [WRITER-A] COMMIT partition=1 offset=4 (next offset to read: 5)
00:33:02.053 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=5
00:33:02.053 [WRITER-A] correlation_id=c244d3ba-2981-4a04-a0b3-319a845d4bed
00:33:03.343 [WRITER-A] SEND to s0932_drafts partition=0 offset=14 correlation_id=c244d3ba-2981-4a04-a0b3-319a845d4bed
00:33:03.343 [WRITER-A] draft=A transitive dependency is an indirect dependency introduced through a direct dependency in a software project.
 A transitive dependency vulnerability occurs when a vulnerable library is included via another library, without the developer explicitly kno
wing about it. This can happen when a dependency's sub-dependency contains a security flaw. The potential risks include unexpected security b
reaches, data leaks, and system instability, as the vulnerability may be exploited through the indirect dependency, compromising the entire a
pplication. These vulnerabilities are often difficult to detect and mitigate without proper dependency management tools.
00:33:03.345 [WRITER-A] COMMIT partition=1 offset=5 (next offset to read: 6)
00:33:04.541 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=9
00:33:04.541 [WRITER-A] correlation_id=c36859d9-c830-4ff5-9494-2609e26ae128
00:33:05.741 [WRITER-A] SEND to s0932_drafts partition=0 offset=15 correlation_id=c36859d9-c830-4ff5-9494-2609e26ae128
00:33:05.741 [WRITER-A] draft=Run `npm audit` to identify known vulnerabilities in your project's dependencies. Review the audit report to se
e which packages are flagged as vulnerable. Check the specific versions of these packages in your `package.json` to determine if they are cur
rently in use. If a vulnerable version is present, consider updating to a secure version using `npm update` or specifying a safe version in t
he `package.json`. Always verify the fix with another `npm audit` to ensure the issue is resolved.
00:33:05.743 [WRITER-A] COMMIT partition=0 offset=9 (next offset to read: 10)
00:33:06.570 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=6
00:33:06.570 [WRITER-A] correlation_id=234283cf-1048-43d1-814b-9df08007235e
00:33:08.091 [WRITER-A] SEND to s0932_drafts partition=0 offset=16 correlation_id=234283cf-1048-43d1-814b-9df08007235e
```

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\reviewer.py
00:31:32.668 REVIEWER Listening on s0932_drafts...
00:31:32.669 REVIEWER RECEIVE from s0932_drafts partition=0 offset=8 correlation_id=30da83a4-f7c2-4eec-be26-d0ba9d63cdc6
00:31:33.635 REVIEWER SEND to s0932_final partition=0 offset=8 correlation_id=30da83a4-f7c2-4eec-be26-d0ba9d63cdc6
00:31:33.635 REVIEWER status=approved review=APPROVED
The draft clearly outlines the necessary steps to address CVE-2021-23337 in lodash 4.17.20, including verification, impact assessment, mitiga
tion strategies, and prioritization of patching.
00:32:50.822 REVIEWER RECEIVE from s0932_drafts partition=0 offset=9 correlation_id=3112eb1f-5e9e-43c2-89ec-3e4a1fcdcc1d
00:32:51.457 REVIEWER SEND to s0932_final partition=0 offset=9 correlation_id=3112eb1f-5e9e-43c2-89ec-3e4a1fcdcc1d
00:32:51.458 REVIEWER status=approved review=APPROVED
The draft clearly outlines the necessary steps to address CVE-2021-23337 in lodash 4.17.20, including verification, impact assessment, updati
ng, and mitigation strategies.
00:32:54.569 REVIEWER RECEIVE from s0932_drafts partition=0 offset=10 correlation_id=6a28e710-02d4-4367-a888-09f692d5c12d
00:32:55.111 REVIEWER SEND to s0932_final partition=0 offset=10 correlation_id=6a28e710-02d4-4367-a888-09f692d5c12d
00:32:55.111 REVIEWER status=approved review=APPROVED
The draft correctly identifies the version of `requests` that fixes CVE-2018-18074 and provides accurate information about the fix and recomm
ended action.
00:32:56.493 REVIEWER RECEIVE from s0932_drafts partition=0 offset=11 correlation_id=2224b266-721a-4495-8344-a6c327484b52
00:32:56.843 REVIEWER SEND to s0932_final partition=0 offset=11 correlation_id=2224b266-721a-4495-8344-a6c327484b52
00:32:56.843 REVIEWER status=approved review=APPROVED
The draft correctly identifies the CVSS severity rating for a score of 9.8 as "Critical."
00:32:58.591 REVIEWER RECEIVE from s0932_drafts partition=0 offset=12 correlation_id=2f57162c-5d93-40fd-bf7e-cdcd1853cbe3
00:32:58.972 REVIEWER SEND to s0932_final partition=0 offset=12 correlation_id=2f57162c-5d93-40fd-bf7e-cdcd1853cbe3
00:32:58.972 REVIEWER status=approved review=APPROVED
The draft clearly and accurately defines an SBOM, explaining its purpose, components, and benefits in a concise manner.
00:33:00.977 REVIEWER RECEIVE from s0932_drafts partition=0 offset=13 correlation_id=6d8a9dde-d6fa-40f3-bf4a-054d0e4dadcb
00:33:01.453 REVIEWER SEND to s0932_final partition=0 offset=13 correlation_id=6d8a9dde-d6fa-40f3-bf4a-054d0e4dadcb
00:33:01.453 REVIEWER status=approved review=APPROVED
The draft correctly identifies the Log4j version that fixes the Log4Shell vulnerability and provides accurate information about the recommend
ed minimum version.
00:33:03.343 REVIEWER RECEIVE from s0932_drafts partition=0 offset=14 correlation_id=c244d3ba-2981-4a04-a0b3-319a845d4bed
00:33:03.779 REVIEWER SEND to s0932_final partition=0 offset=14 correlation_id=c244d3ba-2981-4a04-a0b3-319a845d4bed
00:33:03.779 REVIEWER status=approved review=APPROVED
The draft clearly and accurately explains what a transitive dependency vulnerability is, including its causes, risks, and challenges in detec
tion and mitigation.
00:33:05.742 REVIEWER RECEIVE from s0932_drafts partition=0 offset=15 correlation_id=c36859d9-c830-4ff5-9494-2609e26ae128
00:33:06.049 REVIEWER SEND to s0932_final partition=0 offset=15 correlation_id=c36859d9-c830-4ff5-9494-2609e26ae128
00:33:06.049 REVIEWER status=approved review=APPROVED
The draft clearly explains how to check for and address vulnerable packages in an npm project.
00:33:08.091 REVIEWER RECEIVE from s0932_drafts partition=0 offset=16 correlation_id=234283cf-1048-43d1-814b-9df08007235e
00:33:08.450 REVIEWER SEND to s0932_final partition=0 offset=16 correlation_id=234283cf-1048-43d1-814b-9df08007235e
00:33:08.450 REVIEWER status=approved review=APPROVED
The draft clearly and accurately explains what prototype pollution is, its potential impacts, and provides relevant examples.
00:33:10.251 REVIEWER RECEIVE from s0932_drafts partition=0 offset=17 correlation_id=5fad987a-c6a1-469f-af49-0efd993b9782
00:33:10.619 REVIEWER SEND to s0932_final partition=0 offset=17 correlation_id=5fad987a-c6a1-469f-af49-0efd993b9782
00:33:10.619 REVIEWER status=approved review=APPROVED
The draft clearly and correctly advises immediate patching of Critical severity vulnerabilities, with specific timeframes and communication g
uidance.
00:33:12.864 REVIEWER RECEIVE from s0932_drafts partition=0 offset=18 correlation_id=db08d31c-caed-4c57-8c09-e6a47df20980
00:33:13.210 REVIEWER SEND to s0932_final partition=0 offset=18 correlation_id=db08d31c-caed-4c57-8c09-e6a47df20980
00:33:13.210 REVIEWER status=approved review=APPROVED
The draft clearly and accurately explains the difference between CVE and CWE, highlighting their distinct purposes and applications.
00:33:15.212 REVIEWER RECEIVE from s0932_drafts partition=0 offset=19 correlation_id=ea0d8ccb-72cd-44c9-b187-bb2edbd88c8b
00:33:15.761 REVIEWER SEND to s0932_final partition=0 offset=19 correlation_id=ea0d8ccb-72cd-44c9-b187-bb2edbd88c8b
00:33:15.761 REVIEWER status=approved review=APPROVED
The draft provides a clear, actionable response to CVE-2021-23337, including steps to confirm impact, apply a patch, and communicate with sta
keholders.
00:33:18.088 REVIEWER RECEIVE from s0932_drafts partition=0 offset=20 correlation_id=d4f62c7d-64fc-4196-b463-e7044c747ad7
00:33:18.604 REVIEWER SEND to s0932_final partition=0 offset=20 correlation_id=d4f62c7d-64fc-4196-b463-e7044c747ad7
00:33:18.604 REVIEWER status=approved review=APPROVED
The draft correctly identifies the origin of the patch request for CVE-2018-18074 and explains the context of the vulnerability and its resol
ution.
00:33:20.294 REVIEWER RECEIVE from s0932_drafts partition=0 offset=21 correlation_id=bb1a9916-0bab-4c7f-ae09-37c01a8652f5
00:33:20.652 REVIEWER SEND to s0932_final partition=0 offset=21 correlation_id=bb1a9916-0bab-4c7f-ae09-37c01a8652f5
00:33:20.652 REVIEWER status=approved review=APPROVED
The draft correctly identifies that a CVSS score of 9.8 falls into the Critical severity level.
00:33:22.645 REVIEWER RECEIVE from s0932_drafts partition=0 offset=22 correlation_id=559aaa0a-86fe-4319-8105-26bd6b0fab1b
00:33:23.095 REVIEWER SEND to s0932_final partition=0 offset=22 correlation_id=559aaa0a-86fe-4319-8105-26bd6b0fab1b
```

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\writer.py
00:33:08.091 [WRITER-A] draft-Prototype pollution is a vulnerability that occurs when an attacker injects malicious data into an object's pro
totype chain, altering the behavior of objects in the application. This typically happens through untrusted input that modifies prototype pro
perties, such as `Object.prototype`.

This vulnerability can lead to unauthorized data manipulation, privilege escalation, and even remote code execution. Attackers can exploit it
 to bypass security controls or inject malicious functions into objects.

For example, an attacker might use `Object.assign` or `JSON.parse` with untrusted input to modify the prototype, allowing them to access or a
lter sensitive data, leading to severe security breaches.
00:33:08.092 [WRITER-A] COMMIT partition=1 offset=6 (next offset to read: 7)
00:33:09.434 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=10
00:33:09.435 [WRITER-A] correlation_id=5fad987a-c6a1-469f-af49-0efd993b9782
00:33:10.250 [WRITER-A] SEND to s0932_drafts partition=0 offset=17 correlation_id=5fad987a-c6a1-469f-af49-0efd993b9782
00:33:10.250 [WRITER-A] draft-Patch a Critical severity vulnerability immediately. Assess the risk and prioritize based on system criticality
 and user impact. Apply the patch within 24-48 hours if a reliable fix is available. Communicate the action to stakeholders promptly to ensur
e transparency and minimize exposure. swift response is essential to prevent exploitation and protect users.
00:33:10.262 [WRITER-A] COMMIT partition=0 offset=10 (next offset to read: 11)
00:33:11.333 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=2
00:33:11.333 [WRITER-A] correlation_id=db08d31c-caed-4c57-8c09-e6a47df20980
00:33:12.864 [WRITER-A] SEND to s0932_drafts partition=0 offset=18 correlation_id=db08d31c-caed-4c57-8c09-e6a47df20980
00:33:12.864 [WRITER-A] draft-A **CVE** (Common Vulnerabilities and Exposures) is an identifier for a specific security vulnerability in soft
ware. It helps track and reference individual security issues.

A **CWE** (Common Weakness Enumeration) is a classification system that identifies and categorizes common software weaknesses, such as buffer
 overflows or insecure API usage.

The key difference is that **CVEs** focus on specific vulnerabilities, while **CWEs** focus on the underlying weaknesses that can lead to vul
nerabilities. CVEs are used for tracking and patching issues, whereas CWEs are used for understanding and preventing weaknesses in software d
esign.
00:33:12.865 [WRITER-A] COMMIT partition=1 offset=7 (next offset to read: 8)
00:33:14.067 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=11
00:33:14.067 [WRITER-A] correlation_id=ea0d8ccb-72cd-44c9-b187-bb2edbd88c8b
00:33:15.212 [WRITER-A] SEND to s0932_drafts partition=0 offset=19 correlation_id=ea0d8ccb-72cd-44c9-b187-bb2edbd88c8b
00:33:15.212 [WRITER-A] draft-To address CVE-2021-23337 affecting lodash 4.17.20, first confirm if your application is impacted by reviewing
 the vulnerability details. Immediately update lodash to a patched version (e.g., 4.17.21 or later) to resolve the issue. After the update, mo
nitor your system for any related problems and inform your users or stakeholders about the patch to ensure transparency and maintain trust.
00:33:15.213 [WRITER-A] COMMIT partition=0 offset=11 (next offset to read: 12)
00:33:16.722 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=12
00:33:16.722 [WRITER-A] correlation_id=d4f62c7d-64fc-4196-b463-e7044c747ad7
00:33:18.087 [WRITER-A] SEND to s0932_drafts partition=0 offset=20 correlation_id=d4f62c7d-64fc-4196-b463-e7044c747ad7
00:33:18.087 [WRITER-A] draft-CVE-2018-18074 affects the Linux kernel's `ip6gre` module. The patch was released by the Linux kernel maintaine
rs, specifically Linus Torvalds and the kernel development team. The request for the patch originated from a vulnerability report by the Nati
onal Vulnerability Database (NVD) and was likely triggered by security researchers or users encountering the issue. The official request was
 not publicly detailed, but the patch was prompted by the discovery of a buffer overflow vulnerability in the IPv6 GRE tunneling code.
00:33:18.089 [WRITER-A] COMMIT partition=0 offset=12 (next offset to read: 13)
00:33:19.250 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=3
00:33:19.250 [WRITER-A] correlation_id=bb1a9916-0bab-4c7f-ae09-37c03a0652f5
00:33:20.294 [WRITER-A] SEND to s0932_drafts partition=0 offset=21 correlation_id=bb1a9916-0bab-4c7f-ae09-37c03a0652f5
00:33:20.294 [WRITER-A] draft-The CVSS score ranges are: Critical (9.0-10.0), High (7.0-8.9), Medium (4.0-6.9), and Low (0.0-3.9). A CVSS sco
re of 9.8 falls within the Critical severity level. Therefore, the correct severity level for a CVSS score of 9.8 is Critical.
00:33:20.295 [WRITER-A] COMMIT partition=1 offset=8 (next offset to read: 9)
00:33:21.386 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=9
00:33:21.386 [WRITER-A] correlation_id=559aaa0a-86fe-4319-8105-26bdd6b0fab1b
00:33:22.644 [WRITER-A] SEND to s0932_drafts partition=0 offset=22 correlation_id=559aaa0a-86fe-4319-8105-26bdd6b0fab1b
00:33:22.644 [WRITER-A] draft-An SBOM (Software Bill of Materials) is a formal document listing all components, libraries, and dependencies i
n a software product. Its purpose is to enhance transparency and security by providing visibility into software composition. Common SBOM form
ats include SPDX, CycloneDX, and SWID, each offering structured data for component tracking. SBOMs help identify and manage vulnerabilities b
y enabling rapid detection of outdated or insecure components, facilitating patching and compliance efforts. They are essential for secure so
ftware development and supply chain risk management.
00:33:22.646 [WRITER-A] COMMIT partition=1 offset=9 (next offset to read: 10)
00:33:23.870 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=10
00:33:23.870 [WRITER-A] correlation_id=afe7c89d-6463-448a-9f98-50d161f22153
00:33:25.233 [WRITER-A] SEND to s0932_drafts partition=0 offset=23 correlation_id=afe7c89d-6463-448a-9f98-50d161f22153
00:33:25.233 [WRITER-A] draft-To address Log4Shell (CVE-2021-44228), upgrade to **Log4j 2.17.1** or later. This version includes the necessar
y patch to mitigate the vulnerability. Check the [official Log4j release notes](https://logging.apache.org/log4j/2.x/release-notes.html) for
 confirmation. After upgrading, verify the change by checking the version in your application's dependencies and testing for the fix. Ensure a
ll dependent libraries and environments are also updated to avoid residual risks.
```

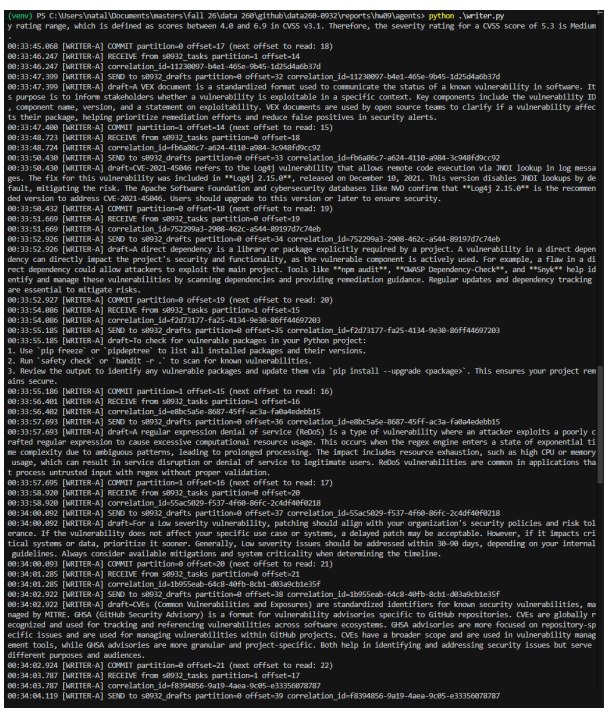
```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\reviewer.py
00:33:23.095 REVIEWER status=approved review=APPROVED
The draft clearly and accurately explains what an SBOM is, its purpose, common formats, and its importance in software security and supply ch
ain management.
00:33:25.233 REVIEWER RECEIVE from s0932_drafts partition=0 offset=23 correlation_id=afe7c89d-6463-448a-9f98-50d161f22153
00:33:25.641 REVIEWER SEND to s0932_final partition=0 offset=23 correlation_id=afe7c89d-6463-448a-9f98-50d161f22153
00:33:25.641 REVIEWER status=approved review=APPROVED
The draft correctly identifies the recommended Log4j version to address Log4Shell and provides clear guidance on verification and additional
 steps.
00:33:27.616 REVIEWER RECEIVE from s0932_drafts partition=0 offset=24 correlation_id=76b737d4-f9cb-4617-bcc0-85980bc41528
00:33:28.002 REVIEWER SEND to s0932_final partition=0 offset=24 correlation_id=76b737d4-f9cb-4617-bcc0-85980bc41528
00:33:28.002 REVIEWER status=approved review=APPROVED
The draft clearly explains what a transitive dependency vulnerability is, its potential impact, and provides actionable steps for mitigation.
00:33:29.715 REVIEWER RECEIVE from s0932_drafts partition=0 offset=25 correlation_id=7097454a-3dc8-40f4-9176-e7b3a70b458d
00:33:30.077 REVIEWER SEND to s0932_final partition=0 offset=25 correlation_id=7097454a-3dc8-40f4-9176-e7b3a70b458d
00:33:30.077 REVIEWER status=approved review=APPROVED
The draft clearly explains how to find and address vulnerable packages in an npm project using standard commands and best practices.
00:33:32.520 REVIEWER RECEIVE from s0932_drafts partition=0 offset=26 correlation_id=f677a965-feee-41bb-977e-98d21953dbea
00:33:32.883 REVIEWER SEND to s0932_final partition=0 offset=26 correlation_id=f677a965-feee-41bb-977e-98d21953dbea
00:33:32.883 REVIEWER status=approved review=APPROVED
The draft clearly and accurately explains the prototype pollution vulnerability, its causes, potential impacts, and prevention methods.
00:33:34.942 REVIEWER RECEIVE from s0932_drafts partition=0 offset=27 correlation_id=50d0aac1-d392-4f78-a63f-40ad81fbdb95
00:33:35.336 REVIEWER SEND to s0932_final partition=0 offset=27 correlation_id=50d0aac1-d392-4f78-a63f-40ad81fbdb95
00:33:35.336 REVIEWER status=approved review=APPROVED
The draft clearly and accurately addresses the question by emphasizing the urgency of fixing Critical vulnerabilities and providing a specifi
c timeframe for action.
00:33:37.407 REVIEWER RECEIVE from s0932_drafts partition=0 offset=28 correlation_id=72d105fb-0f2a-4e43-9437-bea1153d999e
00:33:37.758 REVIEWER SEND to s0932_final partition=0 offset=28 correlation_id=72d105fb-0f2a-4e43-9437-bea1153d999e
00:33:37.758 REVIEWER status=approved review=APPROVED
The draft clearly and accurately explains the difference between CVE and CWE, highlighting their distinct purposes and relationship.
00:33:40.036 REVIEWER RECEIVE from s0932_drafts partition=0 offset=29 correlation_id=e80de4d7-5736-48c3-925a-cba131e82a24
00:33:40.666 REVIEWER SEND to s0932_final partition=0 offset=29 correlation_id=e80de4d7-5736-48c3-925a-cba131e82a24
00:33:40.666 REVIEWER status=approved review=APPROVED
The draft clearly outlines the steps to address CVE-2020-8203 in lodash 4.17.15, including checking for a patched version, updating dependenc
ies, and ensuring compatibility.
00:33:42.715 REVIEWER RECEIVE from s0932_drafts partition=0 offset=30 correlation_id=071fee04-df53-4598-8113-3eb171baf7ee
00:33:43.211 REVIEWER SEND to s0932_final partition=0 offset=30 correlation_id=071fee04-df53-4598-8113-3eb171baf7ee
00:33:43.211 REVIEWER status=approved review=APPROVED
The draft correctly identifies the version of `requests` that fixes CVE-2023-32681 and provides clear guidance for users to upgrade.
00:33:45.067 REVIEWER RECEIVE from s0932_drafts partition=0 offset=31 correlation_id=246a6ece-ac99-4242-95c3-b1b26bdd0e35
00:33:45.551 REVIEWER SEND to s0932_final partition=0 offset=31 correlation_id=246a6ece-ac99-4242-95c3-b1b26bdd0e35
00:33:45.551 REVIEWER status=approved review=APPROVED
The draft correctly identifies the CVSS severity rating for a score of 5.3 as "Medium" based on the CVSS v3.1 scale.
00:33:47.399 REVIEWER RECEIVE from s0932_drafts partition=0 offset=32 correlation_id=11230097-b4e1-465e-9b45-1d25d4a6b37d
00:33:47.805 REVIEWER SEND to s0932_final partition=0 offset=32 correlation_id=11230097-b4e1-465e-9b45-1d25d4a6b37d
00:33:47.805 REVIEWER status=approved review=APPROVED
The draft accurately and clearly defines a VEX document, its purpose, key components, and its use in open source teams.
00:33:50.431 REVIEWER RECEIVE from s0932_drafts partition=0 offset=33 correlation_id=fb6a86c7-a624-4110-a984-3c948fd9cc92
00:33:50.998 REVIEWER SEND to s0932_final partition=0 offset=33 correlation_id=fb6a86c7-a624-4110-a984-3c948fd9cc92
00:33:50.998 REVIEWER status=approved review=APPROVED
The draft correctly identifies Log4j 2.15.0 as the version that fixes CVE-2021-45046 and provides accurate context and recommendations.
00:33:52.926 REVIEWER RECEIVE from s0932_drafts partition=0 offset=34 correlation_id=752299a3-2908-462c-a544-89197d7c74eb
00:33:53.245 REVIEWER SEND to s0932_final partition=0 offset=34 correlation_id=752299a3-2908-462c-a544-89197d7c74eb
00:33:53.245 REVIEWER status=approved review=APPROVED
The draft clearly and accurately defines a direct dependency vulnerability and explains its implications and mitigation strategies.
00:33:55.185 REVIEWER RECEIVE from s0932_drafts partition=0 offset=35 correlation_id=f2d73177-fa25-4134-9e30-86ff44697203
00:33:55.620 REVIEWER SEND to s0932_final partition=0 offset=35 correlation_id=f2d73177-fa25-4134-9e30-86ff44697203
00:33:55.621 REVIEWER status=approved review=APPROVED
The draft correctly outlines the steps to check for vulnerable packages in a Python project and provides clear guidance on how to address the
m.
00:33:57.693 REVIEWER RECEIVE from s0932_drafts partition=0 offset=36 correlation_id=e8bc5a5e-8687-45ff-ac3a-fa0a4edebb15
00:33:58.186 REVIEWER SEND to s0932_final partition=0 offset=36 correlation_id=e8bc5a5e-8687-45ff-ac3a-fa0a4edebb15
00:33:58.186 REVIEWER status=approved review=APPROVED
The draft accurately explains what a ReDoS vulnerability is, its cause, impact, and common context, providing a clear and correct answer to t
he question.
00:34:00.022 REVIEWER RECEIVE from s0932_drafts partition=0 offset=37 correlation_id=55ac5029-f537-4f60-86fc-2c4df40f0218
00:34:00.550 REVIEWER SEND to s0932_final partition=0 offset=37 correlation_id=55ac5029-f537-4f60-86fc-2c4df40f0218
00:34:00.550 REVIEWER status=approved review=APPROVED
```

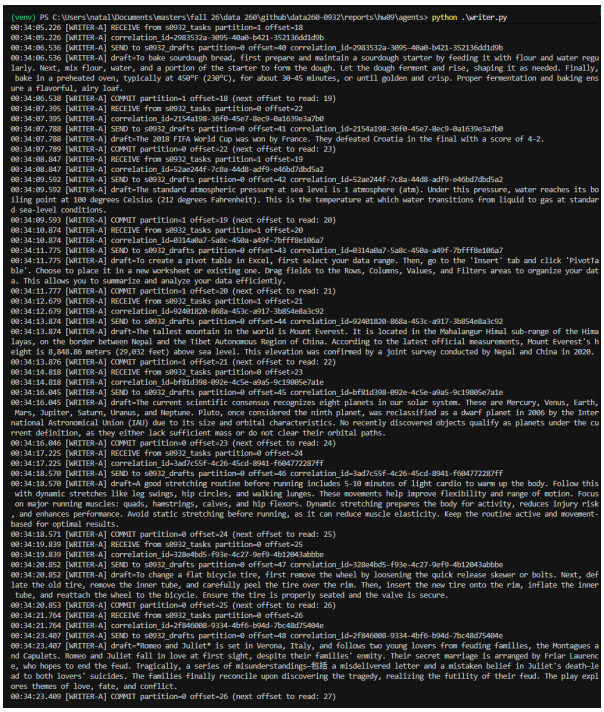
```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\writer.py
00:33:25.234 [WRITER-A] COMMIT partition=1 offset=10 (next offset to read: 11)
00:33:26.360 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=11
00:33:26.360 [WRITER-A] correlation_id=76b737d4-f9cb-4617-bcc0-85980bc41528
00:33:27.616 [WRITER-A] SEND to s0932_drafts partition=0 offset=24 correlation_id=76b737d4-f9cb-4617-bcc0-85980bc41528
00:33:27.616 [WRITER-A] draft-A vulnerability in a transitive dependency means it exists in a package that is indirectly required by the main
 package, not listed as a direct dependency. This happens when the main package depends on another package, which in turn depends on the vuln
erable one. The impact can include security risks like data breaches or system compromise. To address it, update the main package or its depe
ndencies to versions that fix the vulnerability, or use tools like `npm audit` or `snyk` to identify and resolve such issues.
00:33:27.617 [WRITER-A] COMMIT partition=1 offset=11 (next offset to read: 12)
00:33:28.687 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=13
00:33:28.687 [WRITER-A] correlation_id=7097454a-3dc8-40f4-9176-e7b3a70b458d
00:33:29.714 [WRITER-A] SEND to s0932_drafts partition=0 offset=25 correlation_id=7097454a-3dc8-40f4-9176-e7b3a70b458d
00:33:29.714 [WRITER-A] draft-To find vulnerable packages in your npm project, run `npm audit` to scan for known vulnerabilities. This comman
d generates a report showing affected packages and their versions. Review the report to identify specific packages with security issues. Upda
te the vulnerable packages using `npm update` to install the latest secure versions, or manually adjust versions in `package.json` if needed.
 Always ensure dependencies are up to date to mitigate risks.
00:33:29.716 [WRITER-A] COMMIT partition=0 offset=13 (next offset to read: 14)
00:33:30.693 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=12
00:33:30.694 [WRITER-A] correlation_id=f677a965-feee-41bb-977e-98d21953dbea
00:33:32.520 [WRITER-A] SEND to s0932_drafts partition=0 offset=26 correlation_id=f677a965-feee-41bb-977e-98d21953dbea
00:33:32.520 [WRITER-A] draft-Prototype pollution is a vulnerability in JavaScript where an attacker injects malicious properties into the pr
ototype of an object, potentially affecting all instances of that object. This occurs when an application uses `Object.assign()` or `Object.d
efineProperty()` without proper validation, allowing an attacker to overwrite or add properties to the prototype chain.

This can lead to unauthorized data access, method execution, or even remote code execution. Attackers can exploit this to manipulate object b
ehavior or bypass security checks.

For example, an attacker might inject a property like `__proto__.malicious = 'value'`, affecting all objects. Prevention includes validating
 input, avoiding unsafe object assignments, and using libraries like `lodash` with caution. Always sanitize and validate user inputs to avoid
 prototype pollution.
00:33:32.522 [WRITER-A] COMMIT partition=1 offset=12 (next offset to read: 13)
00:33:33.561 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=14
00:33:33.561 [WRITER-A] correlation_id=50d0aac1-d392-4f78-a63f-40ad81fbdb95
00:33:34.942 [WRITER-A] SEND to s0932_drafts partition=0 offset=27 correlation_id=50d0aac1-d392-4f78-a63f-40ad81fbdb95
00:33:34.942 [WRITER-A] draft-The recommended time to fix a Critical vulnerability is **immediately**. Critical vulnerabilities pose signific
ant security risks and should be addressed as a top priority. After assessing the severity, the development team should work swiftly to creat
e and deploy a secure patch. Communication with stakeholders is essential to ensure transparency and timely deployment. Ideally, the fix sho
uld be implemented within **24-48 hours** to minimize exposure and mitigate potential damage. Delaying the fix can lead to data breaches, syst
em compromises, and reputational harm. Prompt action is crucial for maintaining trust and security.
00:33:34.944 [WRITER-A] COMMIT partition=0 offset=14 (next offset to read: 15)
00:33:36.457 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=13
00:33:36.457 [WRITER-A] correlation_id=72d105fb-0f2a-4e43-9437-bea1153d999e
00:33:37.407 [WRITER-A] SEND to s0932_drafts partition=0 offset=28 correlation_id=72d105fb-0f2a-4e43-9437-bea1153d999e
00:33:37.407 [WRITER-A] draft-CVE is a unique identifier for specific cybersecurity vulnerabilities, while CWE is a classification system for
 common weakness types. CVE focuses on individual vulnerabilities in software, whereas CWE identifies the underlying causes leading to those
 vulnerabilities. CVEs often reference CWEs to explain the type of weakness involved, linking specific issues to broader categories of softwar
e flaws. This relationship helps improve understanding and mitigation of security risks.
00:33:37.408 [WRITER-A] COMMIT partition=1 offset=13 (next offset to read: 14)
00:33:38.861 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=15
00:33:38.861 [WRITER-A] correlation_id=e80de4d7-5736-48c3-925a-cba131e82a24
00:33:40.036 [WRITER-A] SEND to s0932_drafts partition=0 offset=29 correlation_id=e80de4d7-5736-48c3-925a-cba131e82a24
00:33:40.036 [WRITER-A] draft-To address CVE-2020-8203 in lodash 4.17.15, first confirm if this version is still used in your project. Then,
 check if a patched version (e.g., 4.17.16 or later) is available and compatible. Update lodash and all dependent packages to the secure versi
on to eliminate the vulnerability. Ensure your dependency tree is fully updated to prevent reintroduction of the issue.
00:33:40.037 [WRITER-A] COMMIT partition=0 offset=15 (next offset to read: 16)
00:33:41.617 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=16
00:33:41.617 [WRITER-A] correlation_id=971fee04-df53-4598-8113-3eb171baf7ee
00:33:42.715 [WRITER-A] SEND to s0932_drafts partition=0 offset=30 correlation_id=071fee04-df53-4598-8113-3eb171baf7ee
00:33:42.715 [WRITER-A] draft-CVE-2023-32681 affects the `requests` package by allowing unauthorized access to certain headers in specific sc
enarios. The vulnerability was addressed in `requests` version 2.29.0. Users should upgrade to **requests 2.29.0 or later** to resolve this i
ssue. This version includes the necessary security patch to prevent unauthorized header access. Always verify the release notes for confirmat
ion.
00:33:42.716 [WRITER-A] COMMIT partition=0 offset=16 (next offset to read: 17)
00:33:44.150 [WRITER-A] RECEIVE from s0932_tasks partition=0 offset=17
00:33:44.151 [WRITER-A] correlation_id=246a6ece-ac99-4242-95c3-b1b26bdd0e35
00:33:45.067 [WRITER-A] SEND to s0932_drafts partition=0 offset=31 correlation_id=246a6ece-ac99-4242-95c3-b1b26bdd0e35
00:33:45.067 [WRITER-A] draft-The CVSS version is typically v3.1 unless specified otherwise. A score of 5.3 falls within the "Medium" severit
```

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\reviewer.py
The draft clearly explains the appropriate timeframe for addressing Low severity vulnerabilities while considering organizational policies, s
ystem criticality, and available mitigations.
00:34:02.922 REVIEWER RECEIVE from s0932_drafts partition=0 offset=38 correlation_id=1b955eab-64c8-40fb-8cb1-d03a9cb1e35f
00:34:03.370 REVIEWER SEND to s0932_final partition=0 offset=38 correlation_id=1b955eab-64c8-40fb-8cb1-d03a9cb1e35f
00:34:03.370 REVIEWER status=approved review=APPROVED
The draft clearly and accurately explains the differences between CVEs and GHSA advisories, highlighting their distinct purposes, scopes, and
 audiences.
00:34:04.119 REVIEWER RECEIVE from s0932_drafts partition=0 offset=39 correlation_id=f8394856-9a19-4aea-9c05-e33356078787
00:34:04.442 REVIEWER SEND to s0932_final partition=0 offset=39 correlation_id=f8394856-9a19-4aea-9c05-e33356078787
00:34:04.442 REVIEWER status=approved review=APPROVED
The draft correctly identifies Canberra as the capital of Australia and provides a clear, concise answer.
00:34:06.536 REVIEWER RECEIVE from s0932_drafts partition=0 offset=40 correlation_id=2983532a-3095-40a0-b421-352136dd1d9b
00:34:06.837 REVIEWER SEND to s0932_final partition=0 offset=40 correlation_id=2983532a-3095-40a0-b421-352136dd1d9b
00:34:06.837 REVIEWER status=approved review=APPROVED
The draft clearly and accurately outlines the steps for baking sourdough bread.
00:34:07.788 REVIEWER RECEIVE from s0932_drafts partition=0 offset=41 correlation_id=2154a198-36f0-45e7-8ec9-0a1639e3a7b0
00:34:08.256 REVIEWER SEND to s0932_final partition=0 offset=41 correlation_id=2154a198-36f0-45e7-8ec9-0a1639e3a7b0
00:34:08.256 REVIEWER status=approved review=APPROVED
The draft correctly and clearly answers the question by stating that France won the 2018 FIFA World Cup and provides the result of the final
 match.
00:34:09.592 REVIEWER RECEIVE from s0932_drafts partition=0 offset=42 correlation_id=52ae244f-7c8a-44d8-adf9-e46bd7dbd5a2
00:34:09.996 REVIEWER SEND to s0932_final partition=0 offset=42 correlation_id=52ae244f-7c8a-44d8-adf9-e46bd7dbd5a2
00:34:09.996 REVIEWER status=approved review=APPROVED
The draft correctly and clearly answers the question by explaining the boiling point of water at sea level under standard atmospheric pressur
e.
00:34:11.775 REVIEWER RECEIVE from s0932_drafts partition=0 offset=43 correlation_id=0314a0a7-5a8c-450a-a49f-7bfff8e106a7
00:34:12.071 REVIEWER SEND to s0932_final partition=0 offset=43 correlation_id=0314a0a7-5a8c-450a-a49f-7bfff8e106a7
00:34:12.071 REVIEWER status=approved review=APPROVED
The draft clearly and correctly explains the steps to create a pivot table in Excel.
00:34:13.875 REVIEWER RECEIVE from s0932_drafts partition=0 offset=44 correlation_id=92401820-868a-453c-a917-3b854e8a3c92
00:34:14.170 REVIEWER SEND to s0932_final partition=0 offset=44 correlation_id=92401820-868a-453c-a917-3b854e8a3c92
00:34:14.170 REVIEWER status=approved review=APPROVED
The draft accurately and clearly answers the question about the tallest mountain in the world.
00:34:16.045 REVIEWER RECEIVE from s0932_drafts partition=0 offset=45 correlation_id=bf81d398-092e-4c5e-a9a5-9c19805e7a1e
00:34:16.443 REVIEWER SEND to s0932_final partition=0 offset=45 correlation_id=bf81d398-092e-4c5e-a9a5-9c19805e7a1e
00:34:16.443 REVIEWER status=approved review=APPROVED
The draft accurately answers the question by stating there are eight planets in the solar system and explains the reclassification of Pluto.
00:34:18.570 REVIEWER RECEIVE from s0932_drafts partition=0 offset=46 correlation_id=3ad7c55f-4c26-45cd-8941-f604772287ff
00:34:18.943 REVIEWER SEND to s0932_final partition=0 offset=46 correlation_id=3ad7c55f-4c26-45cd-8941-f604772287ff
00:34:18.943 REVIEWER status=approved review=APPROVED
The draft provides a clear, accurate, and comprehensive answer to the question about a good stretching routine before running.
00:34:20.852 REVIEWER RECEIVE from s0932_drafts partition=0 offset=47 correlation_id=328e4bd5-f93e-4c27-9ef9-4b12043abbbe
00:34:21.195 REVIEWER SEND to s0932_final partition=0 offset=47 correlation_id=328e4bd5-f93e-4c27-9ef9-4b12043abbbe
00:34:21.195 REVIEWER status=approved review=APPROVED
The draft provides a clear, step-by-step guide for changing a flat bicycle tire without unnecessary complexity.
00:34:23.408 REVIEWER RECEIVE from s0932_drafts partition=0 offset=48 correlation_id=2f046008-9334-4bf6-b94d-7bc48d75404e
00:34:23.797 REVIEWER SEND to s0932_final partition=0 offset=48 correlation_id=2f046008-9334-4bf6-b94d-7bc48d75404e
00:34:23.798 REVIEWER status=approved review=APPROVED
The draft accurately and clearly summarizes the plot of *Romeo and Juliet* with relevant details and themes.
```





Threshold weep table also in metrics.md

threshold	hits	false_hits	hit_rate_%	false_hit_rate_%	mean_latency_ms	total_cost
0.75	26	7	65.0	17.5	770.0	84
0.8	25	7	62.5	17.5	840.1	85
0.85	21	5	52.5	12.5	1080.6	69
0.9	20	5	50.0	12.5	1146.1	70
0.95	14	2	35.0	5.0	1523.3	46

3.3 One false hit in full

Code for false hit

```
    #one false hit
    for threshold in reversed(THRESHOLDS):
        at_threshold = [fh for fh in false_hits_list if fh["threshold"] == threshold]
        if at_threshold:
            print(f"\nOne false hit in full (threshold {threshold})")
            false_hit = at_threshold[0]
            print(f"Incoming query: {false_hit['incoming_query']}")
            print(f"Cached query it matched: {false_hit['cached_query']}")
            print(f"Similarity: {false_hit['similarity']}")
            print(f"Cached answer returned: {false_hit['cached_answer_returned']}")
            print(f"Correct answer: {false_hit['correct_answer']}")
            break
```

Output of one false hit in full

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09> python .\t
One false hit in full (threshold 0.95)
Incoming query: What should our team do about CVE-2020-8203 in lodash 4.17.15?
Cached query it matched: What should our team do about CVE-2021-23337 in lodash 4.17.20?
Similarity: 0.963
Cached answer returned: Verify if your project uses lodash 4.17.20. If so, assess the vulnerability's im
pact on your code. CVE-2021-23337 allows remote code execution, making it critical. Update to a patched
version (e.g., 4.17.21 or later) as soon as possible. If an update isn't feasible, implement a mitigatio
n strategy, such as input sanitization or limiting lodash usage. Prioritize security to prevent potentia
l exploitation.
Correct answer: To address CVE-2020-8203 in lodash 4.17.15, first confirm if this version is still used
in your project. Then, check if a patched version (e.g., 4.17.16 or later) is available and compatible.
Update lodash and all dependent packages to the secure version to eliminate the vulnerability. Ensure yo
ur dependency tree is fully updated to prevent reintroduction of the issue.
```

A short explanation of the likely embedding confusion (why the model considered the two queries similar)
I found that two questions use the same sentence structure where it has “What should our team do about … in lodash …?” The main difference between the two questions is the CVE number and the version number. Because the embedding model captures the meaning of the words, it sees both as the same type of question. I believe the model gives different weights to the different CVE and version numbers, however, these weights are quite small compared to the weights of the wording of the question. This is why the similarity was 0.963. Additionally, the cache returned the answer for the wrong CVE.

3.4 Cost model and threshold choice

Define a cache-miss cost and a false-hit/support cost, and justify both
A cache-miss costs is 1 so that is one full run of the three agents. The false hit is 10 because I think that is a good number to make sure to differentiate between good and bad runs. Additionally, it gives more weight to false hits because in the end that will make or break the model. The total cost will be misses x 1 + false hits x 10.

```
10    MISS_COST = 1
11    FALSE_HIT_COST = 10
```

Code for cost model and threshold choice

```
101     best_row = min(rows, key=lambda r: (r["total_cost"], -r["threshold"]))
102     print(f"\nLowest total cost: threshold {best_row['threshold']} (cost {best_row['total_cost']})")
```

Output of cost model and threshold

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09> python .\t
00:34:24.551 Stored 10 seed answers in the cache

Cost model: total cost = misses x 1 + false hits x 10

Results
Threshold 0.75: hits 26, false hits 7, hit rate 65.0%, false-hit rate 17.5%, mean latency 770.0 ms, tota
l cost 84
Threshold 0.8: hits 25, false hits 7, hit rate 62.5%, false-hit rate 17.5%, mean latency 840.1 ms, total
 cost 85
Threshold 0.85: hits 21, false hits 5, hit rate 52.5%, false-hit rate 12.5%, mean latency 1080.6 ms, tot
al cost 69
Threshold 0.9: hits 20, false hits 5, hit rate 50.0%, false-hit rate 12.5%, mean latency 1146.1 ms, tota
l cost 70
Threshold 0.95: hits 14, false hits 2, hit rate 35.0%, false-hit rate 5.0%, mean latency 1523.3 ms, tota
l cost 46

Lowest total cost: threshold 0.95 (cost 46)
```

Choose one threshold to ship and explain why

I would ship a threshold of 0.95. This is because it had the lowest cost of 46. It has the lowest amount of false hit rates compared to other thresholds. However, the hit rate is lower and mean latency is higher than other thresholds. I think for my domain being vulnerability questions, a wrong answer is worse than a correct one. Therefore, I would go with threshold 0.95.

3.5 Stale answers and invalidation

Code for semantic cache for stale

```python
def handle_query(self, question, context=None):
    log(f"Start: {question}")
    start = time.perf_counter()
    embedding = self.embed(question)
    best, similarity = self.find_most_similar(embedding)
    if (best is not None and similarity >= self.threshold and self.check_data_version and best.data_version != self.get_data_version()):
        log(f"Stale answer (cached data_version {best.data_version}, current {self.get_data_version()}), deleting it")
        self.redis_client.delete(best.id)
        best = None
    if best is not None and similarity >= self.threshold:
        latency_ms = (time.perf_counter() - start)*1000
        log(f"Cache hit, similarity {similarity:.3f}, matched: {best.query}")
        log(f"End: {latency_ms:.1f} ms")
        return {
            "question": question,
            "answer": best.answer,
            "is_cached": True,
            "similarity": round(similarity, 3),
            "cached_query": best.query,
            "data_version": best.data_version,
            "latency_ms": round(latency_ms, 1),
        }
    log(f"Cache miss, best similarity {similarity:.3f}, asking the agents")
    agent_question = question if context is None else f"{question}\n\n{context}"
    result = asyncio.run(ask_agents(agent_question))
    answer = result["answer"]
    self.store(question, embedding, answer)

    latency_ms = (time.perf_counter() - start) * 1000
    log(f"End: {latency_ms:.1f} ms")
    return {
        "question": question,
        "answer": answer,
        "is_cached": False,
        "similarity": round(similarity, 3),
        "cached_query": None,
        "data_version": self.get_data_version(),
        "latency_ms": round(latency_ms, 1),
    }
```

```python
    def bump_data_version(self):
        self.get_data_version()
        return self.redis_client.incr(DATA_VERSION_KEY)
```

Code stale answer and invalidation

```python
from app.database import SessionLocal
from app.models import Vul
from semantic_cache import SemanticResponseCache

SEVERITIES = ["Critical", "High", "Medium", "Low"]

def record_context(vul):
    return (f"Database record from s0932_rel (vulnerabilities table): id={vul.id}, "
            f"package_name={vul.package_name}, vul_code={vul.vul_code}, severity={vul.severity}. "
            f"Answer using this record.")

def update_severity(db, vul, new_severity, cache, bump_version):
    old_severity = vul.severity
    vul.severity = new_severity
    db.commit()
    print(f"Updated s0932_rel vulnerabilities id={vul.id}: severity {old_severity} -> {new_severity}")
    if bump_version:
        print(f"Data version is now {cache.bump_data_version()}")

def show(response):
    print(f"is_cached: {response['is_cached']}")
    print(f"similarity: {response['similarity']}")
    print(f"data_version: {response['data_version']}")
    print(f"answer: {response['answer']}")

def main():
    db = SessionLocal()
    vul = db.query(Vul).order_by(Vul.id.asc()).first()
    original_severity = vul.severity
    other_severities = [s for s in SEVERITIES if s != original_severity]
    question = f"What severity does our database list for {vul.vul_code} in {vul.package_name}?"
    cache = SemanticResponseCache(check_data_version=False)
    cache.clear()
    print("\nirst question (miss, the answer is stored)")
    show(cache.handle_query(question, context=record_context(vul)))
    print("\nupdate the record (no invalidation)")
    update_severity(db, vul, other_severities[0], cache, bump_version=False)
    print("\nsame question again (stale hit)")
    show(cache.handle_query(question, context=record_context(vul)))
    print(f"The database now says severity {vul.severity}, but the cached answer is from before the update.")
    print("\ninvalidation with the data-version marker")
    cache.check_data_version = True
    update_severity(db, vul, other_severities[1], cache, bump_version=True)
    show(cache.handle_query(question, context=record_context(vul)))
    update_severity(db, vul, original_severity, cache, bump_version=True)
    db.close()
```

Output stale answers and invalidation

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09> python .\stale.py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
Loading weights: 100%| 103/103 [00:00<00:00, 5685.88it/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
00:53:43.252 Created vector index s0932:semantic_cache_idx

irst question (miss, the answer is stored)
00:53:43.253 Start: What severity does our database list for 23s67es in nodejs?
00:53:43.263 Cache miss, best similarity 0.000, asking the agents
00:53:43.280 Sent the question to s0932_inbox (correlation_id 2ca6093e-da6d-4121-a173-3abab4f5e37a)
00:53:51.557 Got the answer from s0932_final (correlation_id 2ca6093e-da6d-4121-a173-3abab4f5e37a, status approved)
00:53:51.561 End: 8308.4 ms
is_cached: False
similarity: 0.0
data_version: 1
answer: The vulnerability code `23s67es` is listed in the database record for the `nodejs` package, and the corresponding severity level is `high`.

update the record (no invalidation)
Updated s0932_rel vulnerabilities id=1: severity high -> Critical

same question again (stale hit)
00:53:51.578 Start: What severity does our database list for 23s67es in nodejs?
00:53:51.585 Cache hit, similarity 1.000, matched: What severity does our database list for 23s67es in nodejs?
00:53:51.585 End: 7.2 ms
is_cached: True
similarity: 1.0
data_version: 1
answer: The vulnerability code `23s67es` is listed in the database record for the `nodejs` package, and the corresponding severity level is `high`.
The database now says severity Critical, but the cached answer is from before the update.

invalidation with the data-version marker
Updated s0932_rel vulnerabilities id=1: severity Critical -> High
Data version is now 2
00:53:51.599 Start: What severity does our database list for 23s67es in nodejs?
00:53:51.608 Stale answer (cached data_version 1, current 2), deleting it
00:53:51.609 Cache miss, best similarity 1.000, asking the agents
00:53:51.625 Sent the question to s0932_inbox (correlation_id acbece95-5b3e-4f9d-95d1-76fa32160699)
00:53:51.274 Got the answer from s0932_final (correlation_id acbece95-5b3e-4f9d-95d1-76fa32160699, status approved)
00:53:53.277 End: 1677.8 ms
is_cached: False
similarity: 1.0
data_version: 2
answer: The vulnerability code `23s67es` is listed in the database record for the `nodejs` package, and the corresponding severity level is `High`.
Updated s0932_rel vulnerabilities id=1: severity High -> high
Data version is now 3
```

Output agents planner, writer, and reviewer

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\planner.py
00:53:45.540 PLANNER Listening on s0932_inbox...
00:53:45.540 PLANNER RECEIVE from s0932_inbox partition=0 offset=49 correlation_id=2ca6093e-da6d-4121-a173-3abab4f5e37a
00:53:45.540 PLANNER question=What severity does our database list for 23s67es in nodejs?

Database record from s0932_rel (vulnerabilities table): id=1, package_name=nodejs, vul_code=23s67es, severity=high. Answer using this recor
d.
00:53:48.533 PLANNER SEND to s0932_tasks partition=1 offset=22 correlation_id=2ca6093e-da6d-4121-a173-3abab4f5e37a
00:53:48.533 PLANNER plan=1. Identify the vulnerability code `23s67es` in the database record.
2. Locate the corresponding severity level from the `severity` field.
3. Confirm the severity is listed as `high` for the `nodejs` package.
00:53:51.625 PLANNER RECEIVE from s0932_inbox partition=0 offset=50 correlation_id=acbece95-5b3e-4f9d-95d1-76fa32160699
00:53:51.625 PLANNER question=What severity does our database list for 23s67es in nodejs?

Database record from s0932_rel (vulnerabilities table): id=1, package_name=nodejs, vul_code=23s67es, severity=High. Answer using this recor
d.
00:53:52.314 PLANNER SEND to s0932_tasks partition=1 offset=23 correlation_id=acbece95-5b3e-4f9d-95d1-76fa32160699
00:53:52.314 PLANNER plan=1. Identify the vulnerability code `23s67es` in the database record.
2. Locate the corresponding severity level from the `severity` field.
3. Confirm the severity is listed as `High` for the `nodejs` package.
```

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\writer.py
00:53:49.381 [WRITER-A] Listening on s0932_tasks... mode=AFTER FIX (idempotency, commit after success, 3 retries + dead letter)
00:53:49.382 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=22
00:53:49.382 [WRITER-A] correlation_id=2ca6093e-da6d-4121-a173-3abab4f5e37a
00:53:49.875 [WRITER-A] SEND to s0932_drafts partition=0 offset=49 correlation_id=2ca6093e-da6d-4121-a173-3abab4f5e37a
00:53:49.875 [WRITER-A] draft=The vulnerability code `23s67es` is listed in the database record for the `nodejs` package, and the correspon
ding severity level is `high`.
00:53:49.876 [WRITER-A] COMMIT partition=1 offset=22 (next offset to read: 23)
00:53:52.314 [WRITER-A] RECEIVE from s0932_tasks partition=1 offset=23
00:53:52.314 [WRITER-A] correlation_id=acbece95-5b3e-4f9d-95d1-76fa32160699
00:53:52.774 [WRITER-A] SEND to s0932_drafts partition=0 offset=50 correlation_id=acbece95-5b3e-4f9d-95d1-76fa32160699
00:53:52.775 [WRITER-A] draft=The vulnerability code `23s67es` is listed in the database record for the `nodejs` package, and the correspon
ding severity level is `High`.
00:53:52.777 [WRITER-A] COMMIT partition=1 offset=23 (next offset to read: 24)
```

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\agents> python .\reviewer.py
00:53:51.023 REVIEWER Listening on s0932_drafts...
00:53:51.023 REVIEWER RECEIVE from s0932_drafts partition=0 offset=49 correlation_id=2ca6093e-da6d-4121-a173-3abab4f5e37a
00:53:51.556 REVIEWER SEND to s0932_final partition=0 offset=49 correlation_id=2ca6093e-da6d-4121-a173-3abab4f5e37a
00:53:51.556 REVIEWER status=approved review=APPROVED
The draft correctly identifies the severity level for the vulnerability code `23s67es` in the `nodejs` package based on the provided databa
se record.
00:53:52.775 REVIEWER RECEIVE from s0932_drafts partition=0 offset=50 correlation_id=acbece95-5b3e-4f9d-95d1-76fa32160699
00:53:53.274 REVIEWER SEND to s0932_final partition=0 offset=50 correlation_id=acbece95-5b3e-4f9d-95d1-76fa32160699
00:53:53.274 REVIEWER status=approved review=APPROVED
The draft correctly identifies the severity level for the vulnerability code `23s67es` in the `nodejs` package based on the provided databa
se record.
```

Trade-off of your strategy

My strategy uses one data version number for the whole cache. So when a record changes, the older cached answer will be stale. This is good when the database updates to the record change. However, the one change making all cached answers regardless as stale is a big risk. Therefore, you need every cache hit to allow for Redis to read and check version numbers again.

Part 4: Self-Improving Multi-Agent System

Assignment

reference dataset

reports > hw09 > part4_self_improve_agent > {} reference_data.json > { } 4


```
[
  {
    "cve": "CVE-2021-23337",
    "package": "lodash",
    "vulnerability_type": "command injection",
    "type_keywords": ["command injection", "code injection"],
    "severity": "High",
    "cvss_score": 7.2,
    "fixed_versions": ["4.17.21"]
  },
  {
    "cve": "CVE-2019-10744",
    "package": "lodash",
    "vulnerability_type": "prototype pollution",
    "type_keywords": ["prototype pollution"],
    "severity": "Critical",
    "cvss_score": 9.1,
    "fixed_versions": ["4.17.12"]
  },
  {
    "cve": "CVE-2020-28500",
    "package": "lodash",
    "vulnerability_type": "regular expression denial of service (ReDoS)",
    "type_keywords": ["regular expression denial of service", "redos"],
    "severity": "Medium",
    "cvss_score": 5.3,
    "fixed_versions": ["4.17.21"]
  },
  {
    "cve": "CVE-2018-18074",
    "package": "requests",
    "vulnerability_type": "credential leak (Authorization header sent after an https to http redirect)",
    "type_keywords": ["credential", "authorization header", "information exposure", "information disclosure"],
    "severity": "High",
    "cvss_score": 7.5,
    "fixed_versions": ["2.20.0"]
  },
  {
    "cve": "CVE-2023-32681",
    "package": "requests",
    "vulnerability_type": "credential leak (Proxy-Authorization header)",
    "type_keywords": ["proxy-authorization", "credential", "information exposure", "information disclosure"],
    "severity": "Medium",
    "cvss_score": 6.1,
    "fixed_versions": ["2.31.0"]
  },
  {
    "cve": "CVE-2021-44228",
    "package": "log4j-core",
    "vulnerability_type": "remote code execution (Log4Shell)",
    "type_keywords": ["remote code execution", "rce", "code execution"],
    "severity": "Critical",
    "cvss_score": 10.0
```

Training_data.json

reports > hw09 > part4_self_improve_agent > {} training_data.json > ...

```json
[
  {"id": "train-1", "question": "Which version of lodash fixes CVE-2021-23337?"},
  {"id": "train-2", "question": "What is the severity of CVE-2018-18074 in requests?"},
  {"id": "train-3", "question": "Which version of requests fixes CVE-2023-32681?"},
  {"id": "train-4", "question": "What is the CVSS score of CVE-2019-10744 in lodash?"},
  {"id": "train-5", "question": "What is the capital of Australia?"},
  {"id": "train-6", "question": "What type of vulnerability is CVE-2021-3749 in axios?"},
  {"id": "train-7", "question": "How do I bake sourdough bread?"},
  {"id": "train-8", "question": "Which version of axios fixes CVE-2021-3749?"},
  {"id": "train-9", "question": "What is the boiling point of water at sea level?"}
]
```

Validation_data.json

reports > hw09 > part4_self_improve_agent > {} validation_data.json > ...

```json
[
  {"id": "val-1", "question": "Which version of semver fixes CVE-2022-25883?"},
  {"id": "val-2", "question": "What is the severity of CVE-2021-44228 in log4j-core?"},
  {"id": "val-3", "question": "What is the CVSS score of CVE-2018-18074 in requests?"},
  {"id": "val-4", "question": "Who won the 2018 FIFA World Cup?"},
  {"id": "val-5", "question": "What type of vulnerability is CVE-2020-7598 in minimist?"},
  {"id": "val-6", "question": "Which version of lodash fixes CVE-2020-28500?"},
  {"id": "val-7", "question": "How many planets are in our solar system?"},
  {"id": "val-8", "question": "What is the severity of CVE-2023-32681 in requests?"}
]
```

Test_data.json

reports > hw09 > part4_self_improve_agent > {} test_data.json > ...

```json
[
  {"id": "test-1", "question": "Which version of lodash fixes CVE-2019-10744?"},
  {"id": "test-2", "question": "What is the severity of CVE-2020-28500 in lodash?"},
  {"id": "test-3", "question": "What is the CVSS score of CVE-2021-44228 in log4j-core?"},
  {"id": "test-4", "question": "Which version of requests fixes CVE-2018-18074?"},
  {"id": "test-5", "question": "What type of vulnerability is CVE-2022-25883 in semver?"},
  {"id": "test-6", "question": "What is the tallest mountain in the world?"},
  {"id": "test-7", "question": "What is the severity of CVE-2021-3749 in axios?"},
  {"id": "test-8", "question": "Which version of minimist fixes CVE-2020-7598?"},
  {"id": "test-9", "question": "How do I change a flat bicycle tire?"},
  {"id": "test-10", "question": "What type of vulnerability is CVE-2021-23337 in lodash?"}
]
```

4.1 Repair Failed Answers Using Evaluator Feedback

Code for Answering Agent

```python
def answering_agent(question, lessons, previous_answer=None, feedback=None):
    system_prompt = ANSWERING_PROMPT
    if lessons:
        system_prompt += "\n\nLessons from earlier mistakes (follow them):\n"
        system_prompt += "\n".join(f"- {lesson['correction']}" for lesson in lessons)

    if feedback is None:
        user_prompt = question
    else:
        user_prompt = (f"Question: {question}\n\nYour previous answer: {previous_answer}\n\n"
                       f"Evaluator feedback: {feedback}\n\nWrite a corrected answer.")
    return ask_llm(system_prompt, user_prompt)

```

Code for Evaluator Agent

```python
def evaluator_agent(question, answer, reference):
    cve = find_cve(question)
    record = next((r for r in reference if r["cve"] == cve), None)
    answer_lower = answer.lower()

    #so questions not about vulnerability in the reference dataset aren't answered
    if record is None:
        if REFUSAL.lower().rstrip(".") in answer_lower:
            return {"result": "PASS",
                    "explanation": "The question is not in the reference dataset and the answer correctly declines.",
                    "feedback": "", "failure_pattern": None}
        return {"result": "REVISE",
                "explanation": "The question is not about a vulnerability in the reference dataset, but the answer gives an unsupported answer.",
                "feedback": f'Do not answer this question. Reply exactly: "{REFUSAL}"',
                "failure_pattern": "unsupported_answer"}

    topic = question_topic(question)
    if topic == "fixed_version":
        label = "fixed version"
        expected = " or ".join(record["fixed_versions"])
        correct = any(version in answer for version in record["fixed_versions"])
    elif topic == "severity":
        label = "severity"
        expected = record["severity"]
        correct = re.search(rf"\b{record['severity'].lower()}\b", answer_lower) is not None
    elif topic == "cvss_score":
        label = "CVSS score"
        expected = str(record["cvss_score"])
        numbers = re.findall(r"\d+(?:\.\d+)?", answer)
        correct = any(float(number) == record["cvss_score"] for number in numbers)
    else:
        label = "vulnerability type"
        expected = record["vulnerability_type"]
        correct = any(keyword in answer_lower for keyword in record["type_keywords"])

    if correct:
        return {"result": "PASS",
                "explanation": f"The answer gives the correct {label} ({expected}) from the reference dataset.",
                "feedback": "", "failure_pattern": None}
    return {"result": "REVISE",
            "explanation": f"The answer does not give the correct {label}. The reference dataset says: {expected}.",
            "feedback": f"State the {label} of {record['cve']} in {record['package']}: {expected}.",
            "failure_pattern": f"missing_{topic}"}
```

Code for calling revision

```python
def run_question(item, reference, use_repair, lessons, first_answer=None):
    question = item["question"]
    start = time.perf_counter()
    used = relevant_lessons(question, lessons)
    for lesson in used:
        lesson["use_count"] += 1

    if first_answer is not None:
        answer = first_answer
    else:
        answer = answering_agent(question, used)
    evaluation = evaluator_agent(question, answer, reference)
    attempts = [{"answer": answer, "evaluation": evaluation}]
    if use_repair and evaluation["result"] == "REVISE" and len(attempts) < MAX_ATTEMPTS:
        answer = answering_agent(question, used, previous_answer=answer, feedback=evaluation["feedback"])
        evaluation = evaluator_agent(question, answer, reference)
        attempts.append({"answer": answer, "evaluation": evaluation})
    return {
        "id": item["id"],
        "question": question,
        "lessons_used": [lesson["lesson_id"] for lesson in used],
        "attempts": attempts,
        "number_of_attempts": len(attempts),
        "final_result": evaluation["result"],
        "final_answer": answer,
        "first_failure_pattern": attempts[0]["evaluation"]["failure_pattern"],
        "latency_ms": round((time.perf_counter() - start) * 1000, 1),
    }
```

At least one controlled test case

```
    print("Controlled test case (4.1): the first answer is intentionally incomplete")
    run = run_question(training[0], reference, use_repair=True, lessons=[],
                       first_answer="Upgrade lodash to a newer version.")
```

Output





```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\part4_self_improve_agent> python .\train.py
Final evaluation: PASS
Final response: The vulnerability CVE-2021-3749 is fixed in axios version 0.21.2 and later. It is recommended to upgrade to the latest stable version of axi
os to ensure this vulnerability is addressed.
Latency: 1177.7 ms
New lesson L2 (failure pattern seen 2 times): When asked which version fixes a CVE, give one exact version number and say to upgrade to that version or late
r.

Question (train-9): What is the boiling point of water at sea level?
Lessons loaded: L1
Attempt 1 answer: I can only answer questions about open source package vulnerabilities.
Evaluator result: PASS
Evaluator explanation: The question is not in the reference dataset and the answer correctly declines.
Number of attempts: 1
Final evaluation: PASS
Final response: I can only answer questions about open source package vulnerabilities.
Latency: 171.1 ms

Failure patterns in training: {'missing_fixed_version': 2, 'missing_cvss_score': 1, 'unsupported_answer': 2, 'missing_vulnerability_type': 1}
Saved 2 lesson(s) in lessons.json
L1: The answer responds to a question that is not about open source package vulnerabilities. (use_count 1)
L2: The answer does not give the exact fixed version. (use_count 0)
```

4.2 Save and Reuse Lessons

Code for Evaluator Agent identifies a repeated failure pattern

```python
def create_lesson(pattern, lessons):
    failure_pattern, correction = LESSON_TEMPLATES[pattern]
    lesson = {
        "lesson_id": f"L{len(lessons) + 1}",
        "failure_pattern": failure_pattern,
        "correction": correction,
        "created_at": date.today().isoformat(),
        "use_count": 0,
        "pattern": pattern,
        "topic": "unrelated" if pattern == "unsupported_answer" else pattern.replace("missing_", ""),
    }
    lessons.append(lesson)
    return lesson

def relevant_lessons(question, lessons):
    return [lesson for lesson in lessons if lesson["topic"] == question_topic(question)]
```

Code for Answering Agent must load relevant lessons and use them when producing the answer

```python
def answering_agent(question, lessons, previous_answer=None, feedback=None):
    system_prompt = ANSWERING_PROMPT
    if lessons:
        system_prompt += "\n\nLessons from earlier mistakes (follow them):\n"
        system_prompt += "\n".join(f"- {lesson['correction']}" for lesson in lessons)
```

Demonstrate that a lesson created after one failed answer is loaded and reused for a later related question

```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\part4_self_improve_agent> python .\train.py
Final evaluation: PASS
Final response: The vulnerability CVE-2021-3749 is fixed in axios version 0.21.2 and later. It is recommended to upgrade to the latest stable version of axi
os to ensure this vulnerability is addressed.
Latency: 1177.7 ms
New lesson L2 (failure pattern seen 2 times): When asked which version fixes a CVE, give one exact version number and say to upgrade to that version or late
r.

Question (train-9): What is the boiling point of water at sea level?
Lessons loaded: L1
Attempt 1 answer: I can only answer questions about open source package vulnerabilities.
Evaluator result: PASS
Evaluator explanation: The question is not in the reference dataset and the answer correctly declines.
Number of attempts: 1
Final evaluation: PASS
Final response: I can only answer questions about open source package vulnerabilities.
Latency: 171.1 ms

Failure patterns in training: {'missing_fixed_version': 2, 'missing_cvss_score': 1, 'unsupported_answer': 2, 'missing_vulnerability_type': 1}
Saved 2 lesson(s) in lessons.json
L1: The answer responds to a question that is not about open source package vulnerabilities. (use_count 1)
L2: The answer does not give the exact fixed version. (use_count 0)
```

Lessons.json


```
reports > hw09 > part4_self_improve_agent > {} lessons.json > ...
[
  {
    "lesson_id": "L1",
    "failure_pattern": "The answer responds to a question that is not about open source package vulnerabilities.",
    "correction": "If the question is not about an open source package vulnerability, do not answer it. Reply exactly: \"I can only answer questions about open source package vulnerabilities.\"",
    "created_at": "2026-11-09",
    "use_count": 7,
    "pattern": "unsupported_answer",
    "topic": "unrelated",
    "validated": true,
    "validation_result": "fixed 2, regressions 0"
  },
  {
    "lesson_id": "L2",
    "failure_pattern": "The answer does not give the exact fixed version.",
    "correction": "When asked which version fixes a CVE, give one exact version number and say to upgrade to that version or later.",
    "created_at": "2026-11-09",
    "use_count": 8,
    "pattern": "missing_fixed_version",
    "topic": "fixed_version",
    "validated": true,
    "validation_result": "fixed 0, regressions 0"
  }
]
```


4.3 Validate Improvements Before Keeping Them

Code validate

reports > hw09 > part4_self_improve_agent > validate.py > run_set

```python
from part4_agents import load_json, load_lessons, question_topic, run_question, save_lessons

def run_set(items, reference, lessons):
    results = {}
    for item in items:
        results[item["id"]] = run_question(item, reference, use_repair=False, lessons=lessons)
    return results

def main():
    reference = load_json("reference_data.json")
    validation = load_json("validation_data.json")
    lessons = load_lessons()
    if not lessons:
        print("lessons.json has no lessons. Run train.py first.")
        return
    print("Validation before (no lessons)")
    before = run_set(validation, reference, [])
    for lesson in lessons:
        print(f"\nValidating lesson {lesson['lesson_id']}: {lesson['correction']}")
        after = run_set(validation, reference, [lesson])

        fixed = 0
        regressions = 0
        for item in validation:
            result_before = before[item["id"]]["final_result"]
            result_after = after[item["id"]]["final_result"]
            if result_before == "REVISE" and result_after == "PASS":
                fixed += 1
            if result_before == "PASS" and result_after == "REVISE":
                regressions += 1
            print(f"{item['id']} ({question_topic(item['question'])}): before {result_before}, after {result_after}")

        lesson["validated"] = regressions == 0
        lesson["validation_result"] = f"fixed {fixed}, regressions {regressions}"
        if lesson["validated"]:
            print(f"Keep {lesson['lesson_id']}: fixed {fixed}, regressions {regressions}")
        else:
            print(f"Remove {lesson['lesson_id']}: fixed {fixed}, regressions {regressions} (it made a correct answer fail)")
    save_lessons(lessons)
    print("\nSaved the validation result for each lesson in lessons.json")

if __name__ == "__main__":
    main()
```

Output validate

```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\part4_self_improve_agent> python .\validate.py
Validation before (no lessons)

Validating lesson L1: If the question is not about an open source package vulnerability, do not answer it. Reply exactly: "I can only answer questions about
 open source package vulnerabilities."
val-1 (fixed_version): before REVISE, after REVISE
val-2 (severity): before PASS, after PASS
val-3 (cvss_score): before REVISE, after REVISE
val-4 (unrelated): before REVISE, after PASS
val-5 (vulnerability_type): before REVISE, after REVISE
val-6 (fixed_version): before REVISE, after REVISE
val-7 (unrelated): before REVISE, after PASS
val-8 (severity): before PASS, after PASS
Keep L1: fixed 2, regressions 0

Validating lesson L2: When asked which version fixes a CVE, give one exact version number and say to upgrade to that version or later.
val-1 (fixed_version): before REVISE, after REVISE
val-2 (severity): before PASS, after PASS
val-3 (cvss_score): before REVISE, after REVISE
val-4 (unrelated): before REVISE, after REVISE
val-5 (vulnerability_type): before REVISE, after REVISE
val-6 (fixed_version): before REVISE, after REVISE
val-7 (unrelated): before REVISE, after REVISE
val-8 (severity): before PASS, after PASS
Keep L2: fixed 0, regressions 0

Saved the validation result for each lesson in lessons.json
```

4.4 Four System Variants

Code evaluate

reports > hw09 > part4_self_improve_agent > evaluate_test.py > main

```python
import json
import os
from part4_agents import HERE, load_json, load_lessons, run_question, save_lessons

RAW_DIR = os.path.join(HERE, "..", "raw")

def main():
    reference = load_json("reference_data.json")
    test = load_json("test_data.json")
    lessons = load_lessons()
    validated = [lesson for lesson in lessons if lesson.get("validated")]
    variants = [
        ("Baseline", False, []),
        ("Evaluator Repair", True, []),
        ("Lesson Memory", True, lessons),
        ("Validated Improvement", True, validated),
    ]
    rows = []
    all_runs = {}
    for name, use_repair, variant_lessons in variants:
        print(f"\nRunning {name} ({len(variant_lessons)} lesson(s))")
        runs = [run_question(item, reference, use_repair, variant_lessons) for item in test]
        for run in runs:
            print(f"{run['id']}: {run['final_result']} in {run['number_of_attempts']} attempt(s), lessons {run['lessons_used']}")
        correct = sum(1 for run in runs if run["final_result"] == "PASS")
        rows.append({
            "variant": name,
            "correct": correct,
            "incorrect": len(runs) - correct,
            "average_attempts": round(sum(run["number_of_attempts"] for run in runs) / len(runs), 2),
            "average_latency_ms": round(sum(run["latency_ms"] for run in runs) / len(runs), 1),
        })
        all_runs[name] = runs

    save_lessons(lessons)
    os.makedirs(RAW_DIR, exist_ok=True)
    with open(os.path.join(RAW_DIR, "part4_test_results.json"), "w") as f:
        json.dump({"table": rows, "runs": all_runs}, f, indent=2)
    print(f"\nResults on test_data.json ({len(test)} questions)")
    for row in rows:
        print(f"{row['variant']}: correct {row['correct']}, incorrect {row['incorrect']}, "
              f"average attempts {row['average_attempts']}, average latency {row['average_latency_ms']} ms")
    print("\nSaved raw/part4_test_results.json")

if __name__ == "__main__":
    main()

```

Output evaluate

```
Saved the validation result for each lesson in lessons.json
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw09\part4_self_improve_agent> python .\evaluate_test.py

Running Baseline (0 lesson(s))
test-1: PASS in 1 attempt(s), lessons []
test-2: REVISE in 1 attempt(s), lessons []
test-3: PASS in 1 attempt(s), lessons []
test-4: REVISE in 1 attempt(s), lessons []
test-5: REVISE in 1 attempt(s), lessons []
test-6: REVISE in 1 attempt(s), lessons []
test-7: PASS in 1 attempt(s), lessons []
test-8: REVISE in 1 attempt(s), lessons []
test-9: REVISE in 1 attempt(s), lessons []
test-10: REVISE in 1 attempt(s), lessons []

Running Evaluator Repair (0 lesson(s))
test-1: PASS in 1 attempt(s), lessons []
test-2: PASS in 2 attempt(s), lessons []
test-3: PASS in 1 attempt(s), lessons []
test-4: PASS in 2 attempt(s), lessons []
test-5: PASS in 2 attempt(s), lessons []
test-6: PASS in 2 attempt(s), lessons []
test-7: PASS in 1 attempt(s), lessons []
test-8: PASS in 2 attempt(s), lessons []
test-9: PASS in 2 attempt(s), lessons []
test-10: PASS in 2 attempt(s), lessons []

Running Lesson Memory (2 lesson(s))
test-1: PASS in 2 attempt(s), lessons ['L2']
test-2: PASS in 2 attempt(s), lessons []
test-3: PASS in 1 attempt(s), lessons []
test-4: PASS in 2 attempt(s), lessons ['L2']
test-5: PASS in 2 attempt(s), lessons []
test-6: PASS in 1 attempt(s), lessons ['L1']
test-7: PASS in 1 attempt(s), lessons []
test-8: PASS in 2 attempt(s), lessons ['L2']
test-9: PASS in 1 attempt(s), lessons ['L1']
test-10: PASS in 2 attempt(s), lessons []

Running Validated Improvement (2 lesson(s))
test-1: PASS in 2 attempt(s), lessons ['L2']
test-2: PASS in 2 attempt(s), lessons []
test-3: PASS in 1 attempt(s), lessons []
test-4: PASS in 2 attempt(s), lessons ['L2']
test-5: PASS in 2 attempt(s), lessons []
test-6: PASS in 1 attempt(s), lessons ['L1']
test-7: PASS in 1 attempt(s), lessons []
test-8: PASS in 2 attempt(s), lessons ['L2']
test-9: PASS in 1 attempt(s), lessons ['L1']
test-10: PASS in 2 attempt(s), lessons []

Results on test_data.json (10 questions)
Baseline: correct 3, incorrect 7, average attempts 1.0, average latency 805.1 ms
Evaluator Repair: correct 10, incorrect 0, average attempts 1.7, average latency 1150.3 ms
Lesson Memory: correct 10, incorrect 0, average attempts 1.6, average latency 1006.4 ms
Validated Improvement: correct 10, incorrect 0, average attempts 1.6, average latency 1031.2 ms

Saved raw/part4_test_results.json
```

Completed results table also in metrics.md

Variant	Correct answers	Incorrect answers	Average attempts	Average latency (ms)
Baseline	3	7	1.0	805.1
Evaluator Repair	10	0	1.7	1150.3
Lesson Memory	10	0	1.6	1006.4
Validated Improvement	10	0	1.6	1031.2

A brief explanation of the agent communication, lesson reuse, stopping condition, validation process, and one limitation

The answering agent sends the answer to the evaluation who checks it with the reference_data.json. It then will send a pass or revise along with an explanation on its decision and feedback. In the case where the evaluation agent gives a result of revision, the answering agent will use the feedback to revise the answer.

Lessons are saved to lessons.json when a failure happens 2 times in training. The answering agent loads the lessons first to reference before answering. This allows it to add them to its prompt and attempt to pass an answer.

The stopping condition corresponding to the instructions is 2. So, each question gets 2 attempts and then it will stop after the second attempt.

The validation_data.json is used to test the lessons saved and with no lessons. For the validation process I am checking if the revise becomes passed, which means the lesson fixed the question. I am also

checking if vice versa happens that means regression. Finally, I am checking if the lesson has no effect on the decision. Lessons are only saved if there is no regression. In the end, I am left with two lessons in my lessons.json file.

One limitation is that the lesson only changes the instructions the model has and cannot improve the model's overall performance itself. For example, after lesson 2 is used, I can see that some test results still show that the evaluator agent said it needed revision in order to get the correct answer to the question.

AI_USE.md

